



北京大学
PEKING UNIVERSITY

基于覆盖率增强的模糊测试



汇报人：许嘉星



日期：2025/11/04



- 1/ FunFuzz: Greybox Fuzzing with Function Significance
- 2/ FunFuzz: A Function-Oriented Fuzzer for Smart Contract Vulnerability Detection with High Effectiveness and Efficiency
- 3/ TCP-Fuzz: Detecting Memory and Semantic Bugs in TCP Stacks with Fuzzing
- 4/ PromeFuzz: A Knowledge-Driven Approach to Fuzzing Harness Generation with Large Language Models



PART 01 ▶

FunFuzz: Greybox Fuzzing with Function Significance TOSEM 2025

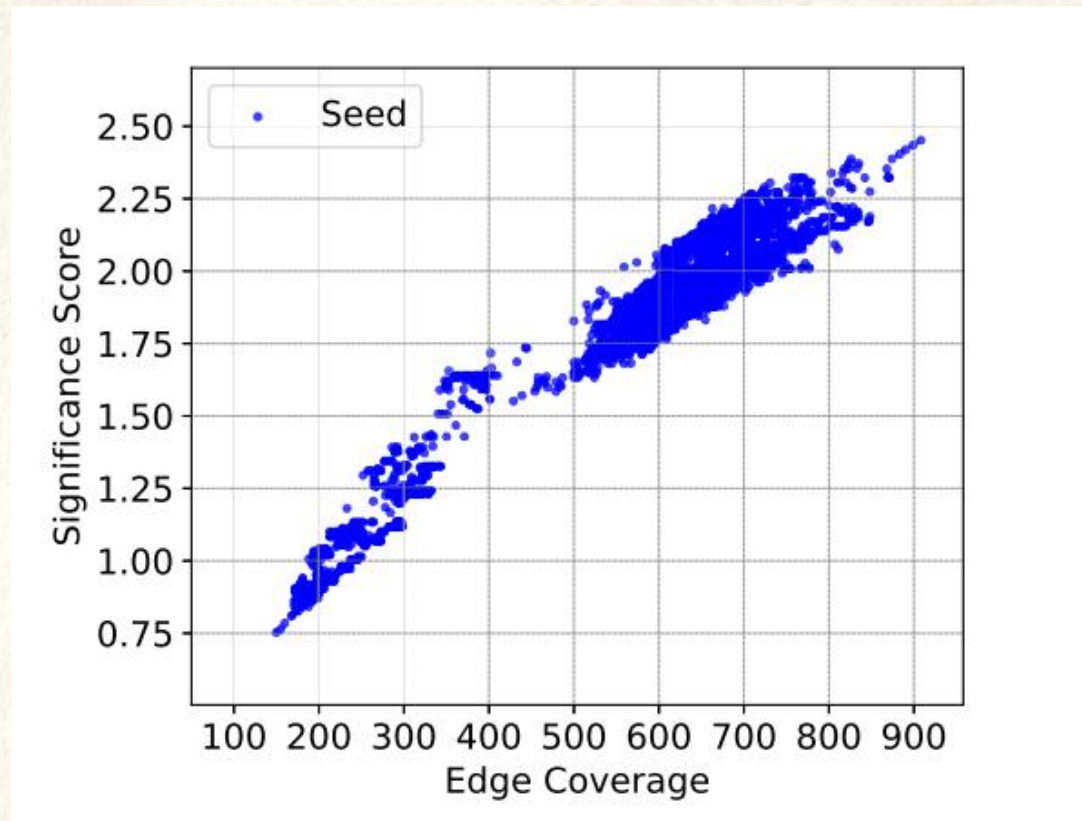
- 灰盒模糊测试技术通过轻量级的程序插桩来收集运行时的代码覆盖信息（如边覆盖率），并以此作为反馈来指导测试输入的生成与选择。
- 具体而言，模糊测试器会持续运行被测试程序，利用初始种子输入生成新的测试用例，一旦发现新的代码覆盖，便将对应输入加入种子队列以进一步演化。在这一过程中，如何为种子分配合适的“能量”（即基于该种子生成新测试用例的次数），即功率调度（Power Scheduling），至关重要。
- 传统方法倾向于为覆盖了更多代码的种子分配更多能量，以期保持其优势。

- 尽管传统灰盒模糊测试非常强大，但其有效性在某些情况下会受到限制。一个关键原因在于，传统方法只关注代码覆盖的“量”，而忽略了被覆盖代码的“质”，即被~~执行函数的重要~~性程度。
- 具体而言，某些种子可能执行了那些在程序调用图中处于中心位置、连接了大量其他函数的“有影响力”的函数，从而更有可能在未来探索到未被覆盖的代码区域；而另一些种子则可能主要执行影响力较小的函数。传统模糊测试器无法识别这种差异

Caller	Callee	First Occur
parse_statement (total: 27)	parse_return	mjs.c:13329
	parse_if	mjs.c:13344
...	...	...
	cs_log_print_prefix	mjs.c:12893
	cs_log_printf	mjs.c:12893
	mjs_set_errorf	mjs.c:12894
	pnext	mjs.c:12894
parse_if (total: 27)	parse_expr	mjs.c:12896
	emit_byte	mjs.c:12898
	emit_init_offset	mjs.c:12898
	parse_block_or_stmt	mjs.c:12903
	mjs_bcode_insert_offset	mjs.c:12920
...	...	...
	mjs_set_errorf	mjs.c:12941
	cs_log_print_prefix	mjs.c:12941
parse_return (total: 7)	cs_log_printf	mjs.c:12941
	pnext	mjs.c:12941
	parse_expr	mjs.c:12941
	emit_byte	mjs.c:12942
...	...	...

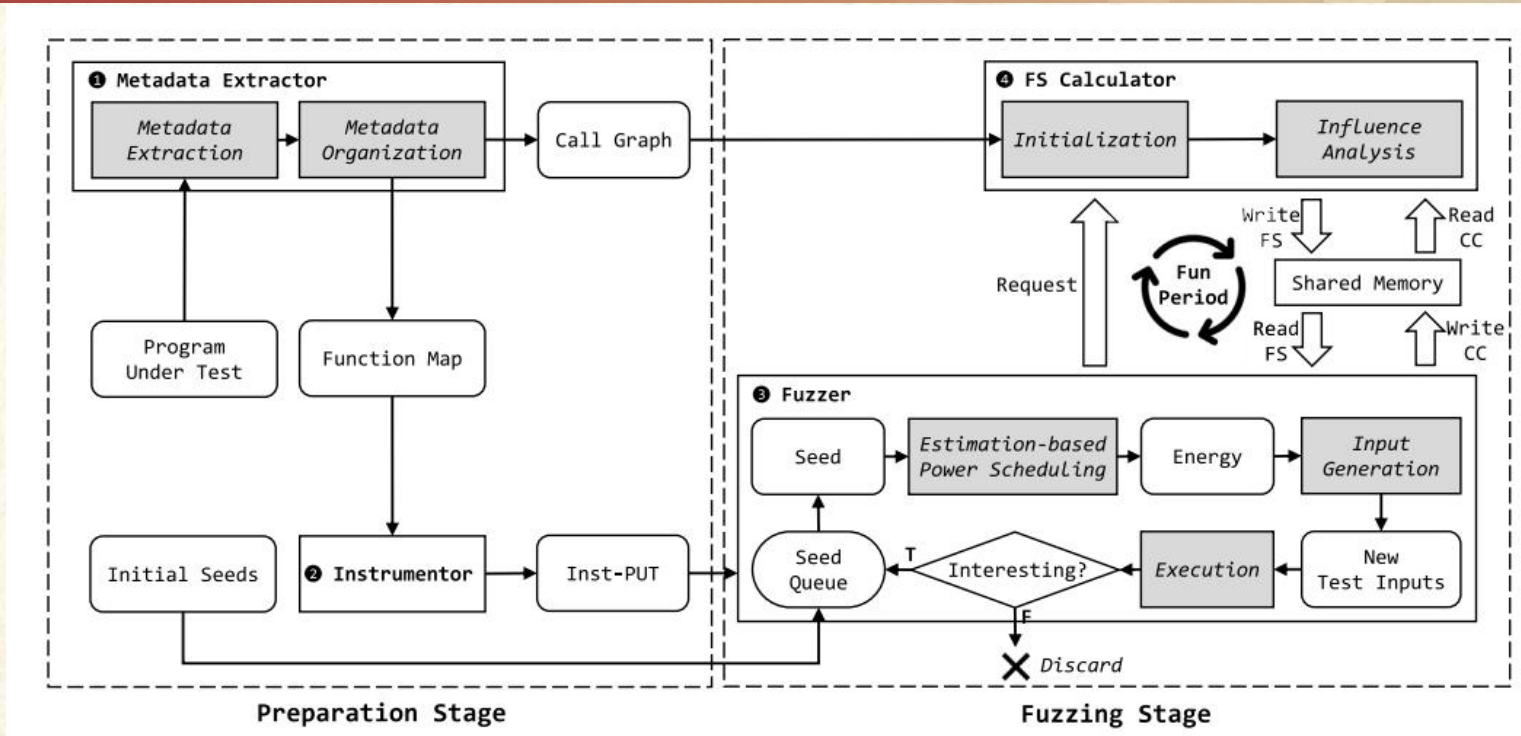
调用关系例子

- 通过使用 AFL++ 对一个真实世界的程序 objdump 进行 24 小时的模糊测试，并通过累积种子覆盖的函数的 FS 值来量化这些能力，即作为**重要性分数**。
- Y 轴表示种子实现的重要性分数，X 轴表示边覆盖情况。如图 1 所示，在 600 的边缘覆盖范围内，显著性得分并非集中在一个小范围内，而是分布在相对较大的范围（1.75 至 2.15）内。这种现象表明，在实际操作中，**种子执行重要功能的能力是极不均衡的**。不幸的是，传统的灰盒式模糊测试器通常并不了解这种不均衡性。



使用AFL++对objdump进行24小时模糊测试时，种子程序获得的显著性分数

1. 函数重要性的量化
2. 对种子重要性得分的动态计算
3. 基于估计的功率调度策略。



该架构包括四个主要组件：元数据提取器（Metadata Extractor）、插桩器（Instrumentor）、模糊测试器（Fuzzer）和FS计算器（FS Calculator）。方法分为准备阶段和模糊测试阶段。在准备阶段，元数据提取器会静态分析被测试程序（PUT），提取函数名和调用关系，构建调用图，并为每个函数分配唯一ID。插桩器则对PUT进行插桩，以便在运行时跟踪函数执行和调用计数。在模糊测试阶段，模糊测试器会周期性地收集到的调用计数传递给FS计算器，用于动态更新FS值，并基于这些值进行种子优化。

论文采用**影响力分析** (Influence Analysis) 来度量函数在调用图中的中心性。

具体来说, 它使用**Katz中心性算法**, 该算法考虑函数节点的邻居重要性以及连接边的权重来计算FS值。

为了更贴合模糊测试的目标, 论文在分析前会反转调用边的方向, 使得影响力能够从被调用函数传播回调用函数, 从而突出那些在调用链中处于主导地位、连接多个其他函数的“枢纽”函数。

边权重的计算基于**动态调用概率**: 对于两个函数之间的调用关系, 权重由其互补调用概率决定, 即 $1 - \text{实际调用概率}$

这样可以优先关注那些执行频率较低但可能触发新路径的调用关系。通过这种方式, FS值能够动态反映函数在测试过程中的实际影响力。

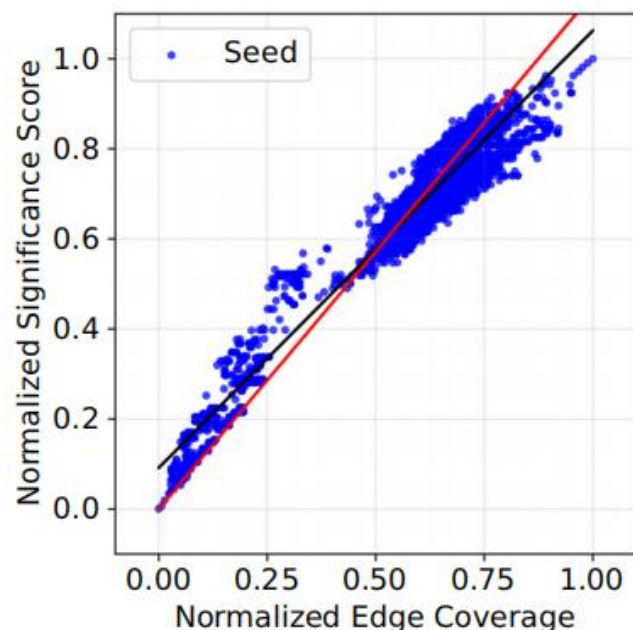
在运行时, FUNFUzz会为每个种子计算**重要性得分** (Significance Score)。这一得分是通过**累加**该种子执行的所有函数的FS值得到的。

具体而言, 模糊测试器会跟踪每个种子的**执行状态**, 记录哪些函数被触发, 然后根据公式将对应函数的FS值汇总。这种打分机制使得种子不仅因覆盖代码的“量”而被评估, 更因其覆盖代码的“质”而被区分。例如, 一个执行了高FS值函数的种子, 即使其边覆盖率与另一个种子相同, 也会获得更高的重要性得分, 从而被视为**更优选的候选**。

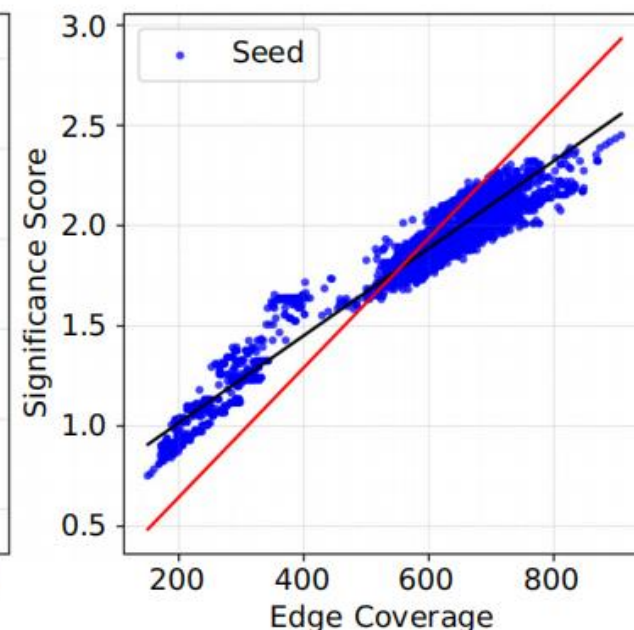
核心技术——基于估计的功率调度

为了**智能调整种子能量**，FUNFUzz构建了一个简单的线性模型来估计种子“应该”达到的重要性得分。该模型以种子的边覆盖率作为自变量，以重要性得分作为因变量，形式为 $y = \mu x$ 。其中估计系数 μ 通过运行时数据的平均值动态计算。论文通过**实证研究**验证了这种线性关系的可行性。

在多个真实世界程序上
种子覆盖率与重要性得分呈现出强正相关性
这种估计模型的优势在于其轻量级
避免了复杂计算对模糊测试吞吐量的影响。
如图所示，该模型在归一化数据上**拟合良好**
支持了其在实际应用中的可靠性。



(a) Estimation with normalized data.



(b) Estimation with original data.

核心技术——基于估计的功率调度

对于每个种子，模糊测试器会计算其实际重要性得分与模型估计得分的比率，并据此生成一个**缩放因子**来调整基础能量（如AFL++默认分配的能量）。

如果种子的实际得分超过估计值（即表现优于预期），缩放因子大于1，能量被放大；反之，则能量被缩减。这种动态调整确保了资源向那些执行了更多重要函数的种子倾斜，从而提高了探索未知代码区域的概率。

此外，方法还引入了“Fun Period”机制，即周期性地更新FS值，并通过**惰性计算**（Lazy FS Computation）和**双共享内存**（Double-SHMs）基础设施来最小化运行时开销，保证模糊测试的吞吐量不受显著影响。

双共享内存：FUNFUzz为两类关键数据——

调用计数（Call Counts, CC）和函数重要性（FS）值分别准备了两块共享内存区域：

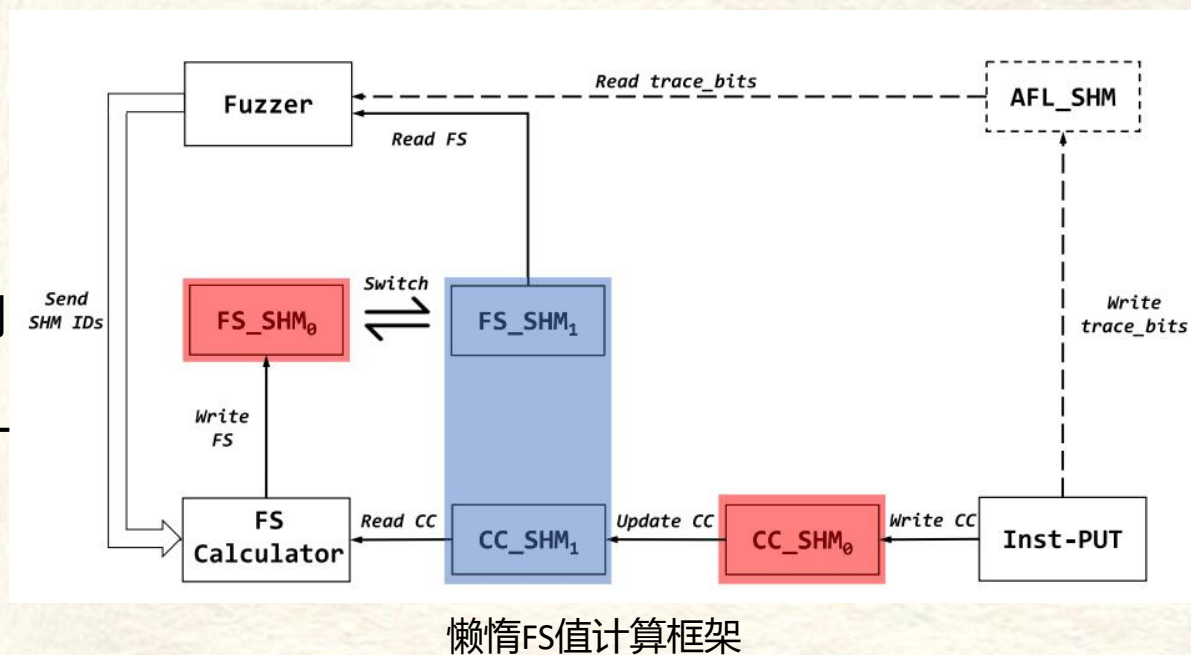
一块称为“**冷**”共享内存，用于读取稳定数据；

另一块称为“**热**”共享内存，用于写入最新数据

对于调用计数（CC）：模糊测试器在运行时将实时统计的调用计数写入热CC SHM。

而当一次计算周期到来时，FS计算器则从冷CC SHM中读取上一阶段积累的稳定数据进行计算。

对于函数重要性（FS）值：FS计算器将计算出的最新FS值写入热FS SHM。而模糊测试器在进行种子评分和功率调度时，则从冷FS SHM中读取上一阶段计算好的、稳定的FS值来使用。



核心技术——惰性FS值计算

惰性FS计算 (Lazy FS Computation) 策略则定义了如何利用双共享内存进行高效、无阻塞的协同工作。

1. **周期触发**: 模糊测试器在主循环中检查时间

每当距离上一次FS更新过去了预设的“Fun Period” (默认为60秒), 便触发一次更新流程。

2. **数据切换与清理**:

首先, 它将热CC SHM中积累的调用计数复制到冷CC SHM中, 作为本次计算的输入数据, 然后清空热CC SHM, 为下一周期计数做准备。

接着, 它交换冷FS SHM和热FS SHM的角色。之前的热FS SHM变成新的冷FS SHM 立即为模糊测试器提供一套可用的FS值。

而之前的冷FS SHM则变为新的热FS SHM并被清空, 准备接收新值。

3. **异步计算**: 模糊测试器通过一个管道 (Pipe), 将新的热FS SHM和冷CC SHM的标识符发送给FS计算器, 然后立即返回并继续其核心的模糊测试工作

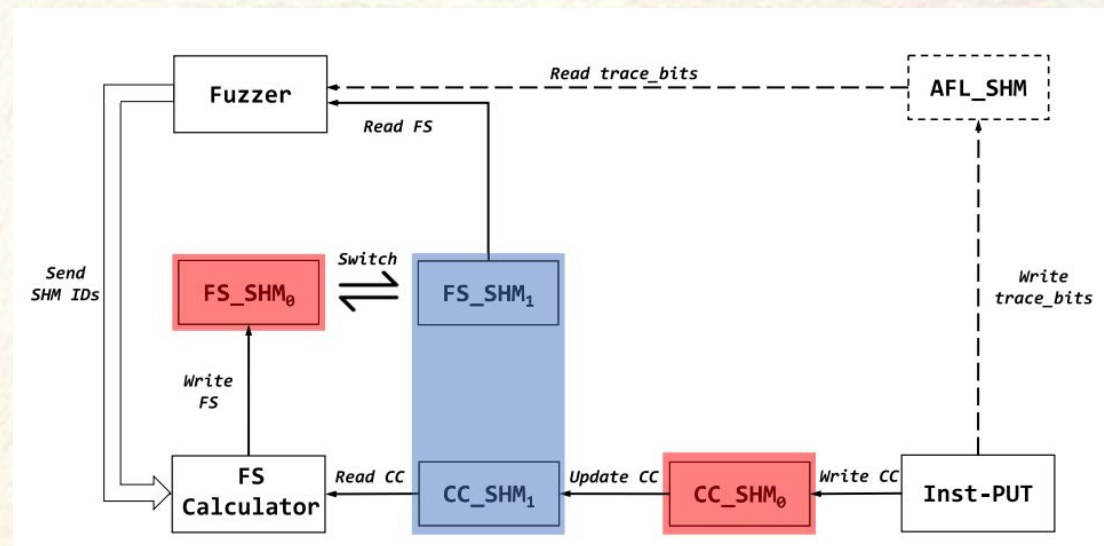
(如执行测试用例、筛选种子等), 而不会等待计算完成。

4. **并行计算**: FS计算器作为一个独立的进程, 在接收到请求后, 从指定的冷CC SHM中读取数据, 执行耗时的影响力分析算法, 计算出新的FS值, 然后写入指定的热FS SHM中。

此时, 这些新值已准备就绪,

将在下一个“Fun Period”到来时被切换为冷FS SHM供模糊测试器使用。

确保了模糊测试器几乎不会因为FS值的计算而陷入等待状态, 从而最大限度地保障了测试吞吐量



惰性FS值计算框架

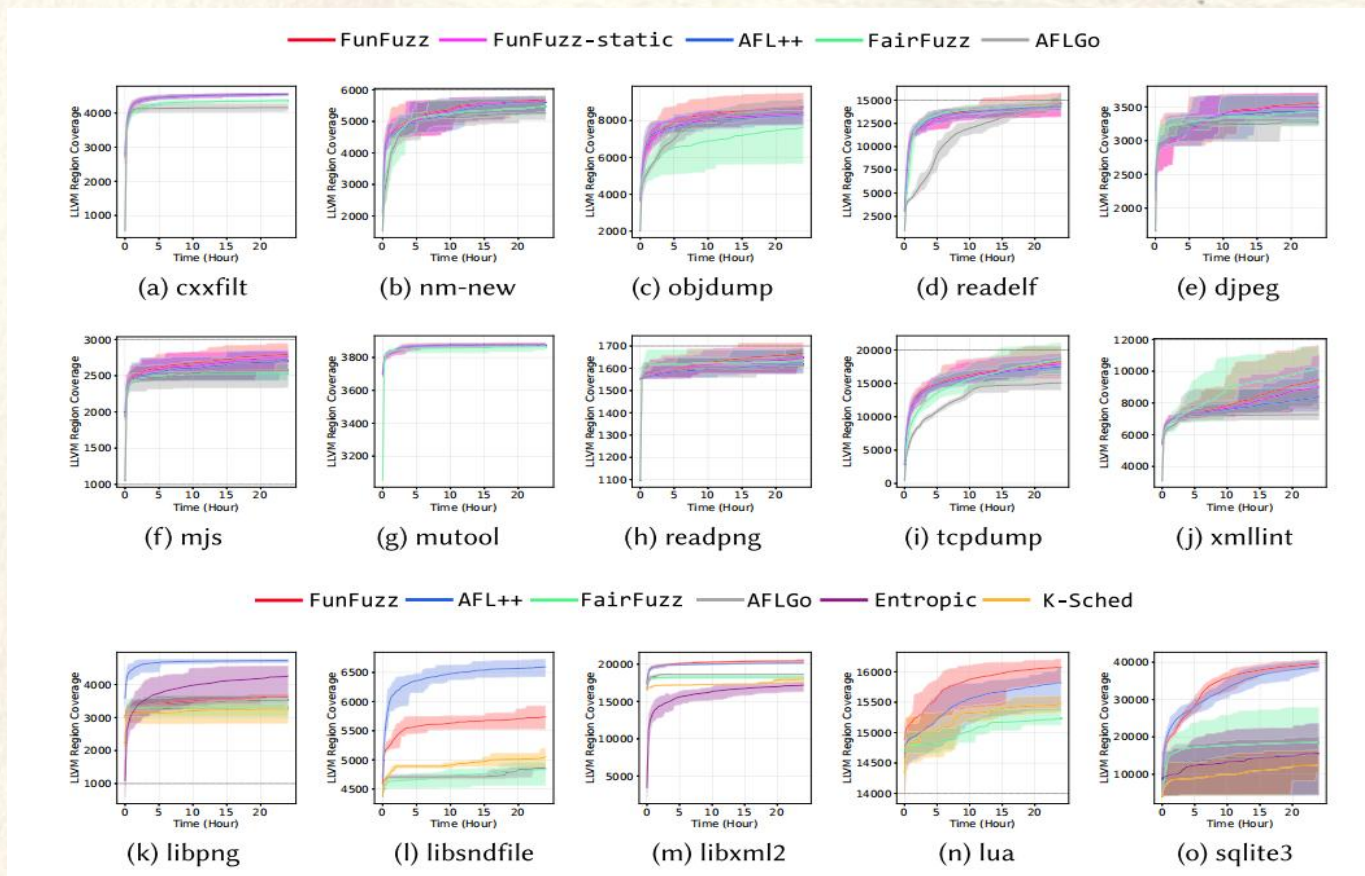
实验结果设置

实验设置涵盖了15个测试目标
10个来自真实世界项目，5个来自Magma基准
7个对比的模糊测试器
包括AFL++、FairFuzz、AFLGo、Entropic和
K-Scheduler等业界先进工具。
总计超过40,800 CPU小时的测试，
并通过20次重复试验以消除随机性
确保了结果的统计可靠性。

	TID	Project	Target & Cmdline	Initial Seeds
Real-world	T01	binutils-2.28	cxxfilt	“_Z1fv”
	T02		nm-new	small_exec.elf
	T03		objdump -d	
	T04		readelf -a	
	T05	libjpeg-turbo-1.5.1	djpeg	not_kitty.jpg
	T06	libpng-1.6.29	readpng	not_kitty.png
	T07	mjs-2.20	mjs -f	small_script.js
	T08	mupdf-1.9	mutool draw	small.pdf
	T09	tcpdump-4.9.0	tcpdump	small_capture.pcap
	T10	libxml2-2.9.4	xmllint	small_document.xml
Magma	T11	libpng	libpng_read_fuzzer	Magma Default [48]
	T12	libsndfile	sndfile_fuzzer	
	T13	libxml2	libxml2_xml_read_memory_fuzzer	
	T14	lua	lua	
	T15	sqlite3	sqlite3_fuzz	

实验结果分析——RQ1 覆盖率表现

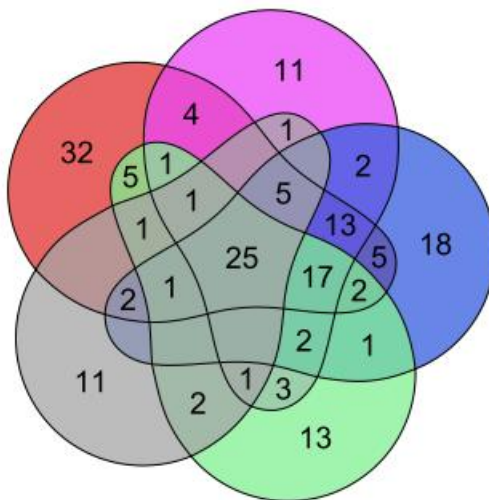
在**代码覆盖**方面，实验结果展现出FUNFUzz的显著优势。
如图所示，在10个真实世界目标上，FUNFUzz在13个目标上均优于其基础版本AFL++
区域覆盖率提升幅度从0.1%到18.4%不等。
尤为值得注意的是，在objdump、readelf等目标上，FUNFUzz取得了大幅度的覆盖提升
这表明其FS导向的调度策略能有效引导测试探索**更广泛**的代码区域。



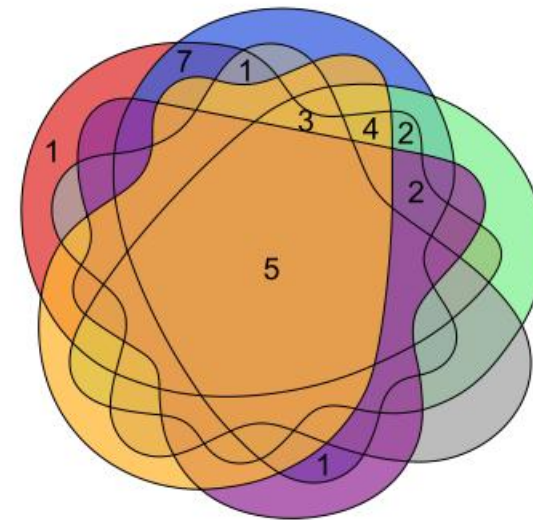
实验结果分析——RQ2 真实漏洞发现

在7个可触发崩溃的真实世界目标上，FUNFUzz共暴露了**114**个独特崩溃，远超AFL++的93个和FairFuzz的98个。更重要的是，FUNFUzz发现了**32**个其它测试器未能触发的崩溃。无论是触发总数还是独特漏洞数均位列**第一**，并成功找到了一个其它所有测试器都遗漏的漏洞。

FunFuzz FunFuzz-static AFL++ FairFuzz AFLGo Entropic K-Sched



(a) Exposed crashes.



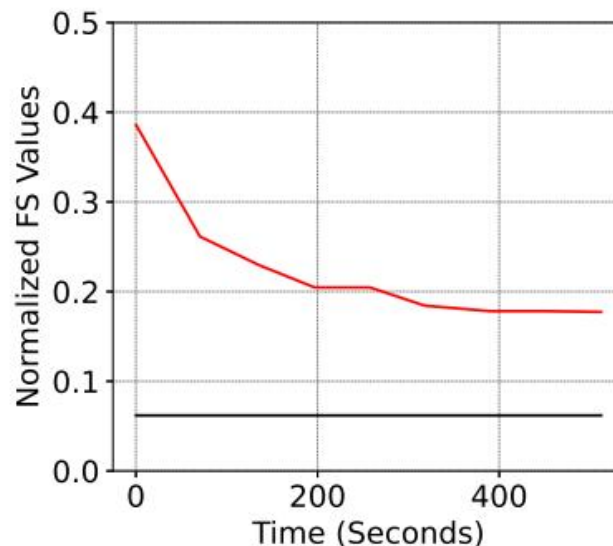
(b) Triggered Magma bugs.

维恩图生动地展示了各测试器在漏洞发现上的重叠与差异：FUNFUzz不仅覆盖了**大部分公共漏洞**，更拥有可观的独有漏洞集合

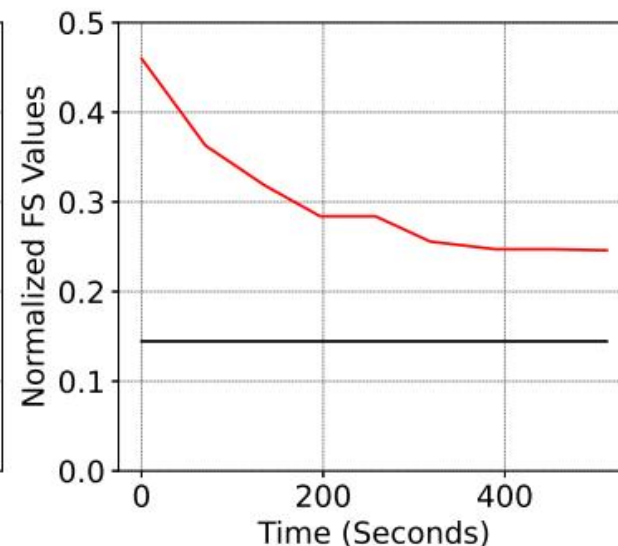
实验结果分析——RQ3 使用动态FS值的好处

与使用静态调用图计算FS值的FUNFuzz-Static配置相比
默认的FUNFUZZ (动态) 在绝大多数目标上取得了更高的代码覆盖和漏洞发现数。

如图所示，函数的FS值在测试过程中会动态波动
这表明基于运行时调用频率的动态更新能更**精确地捕捉**函数真实的重要性
从而做出更优的调度决策。

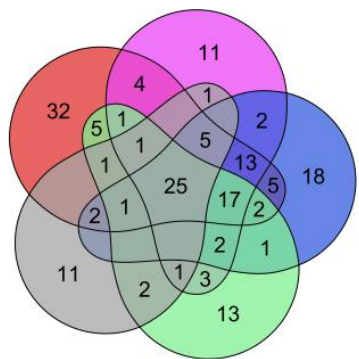


(a) f3: mjs_bcode_insert_off

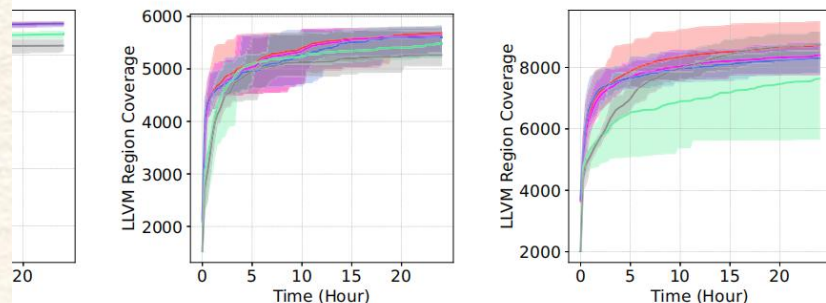


(b) f4: parse_block_or_stmt

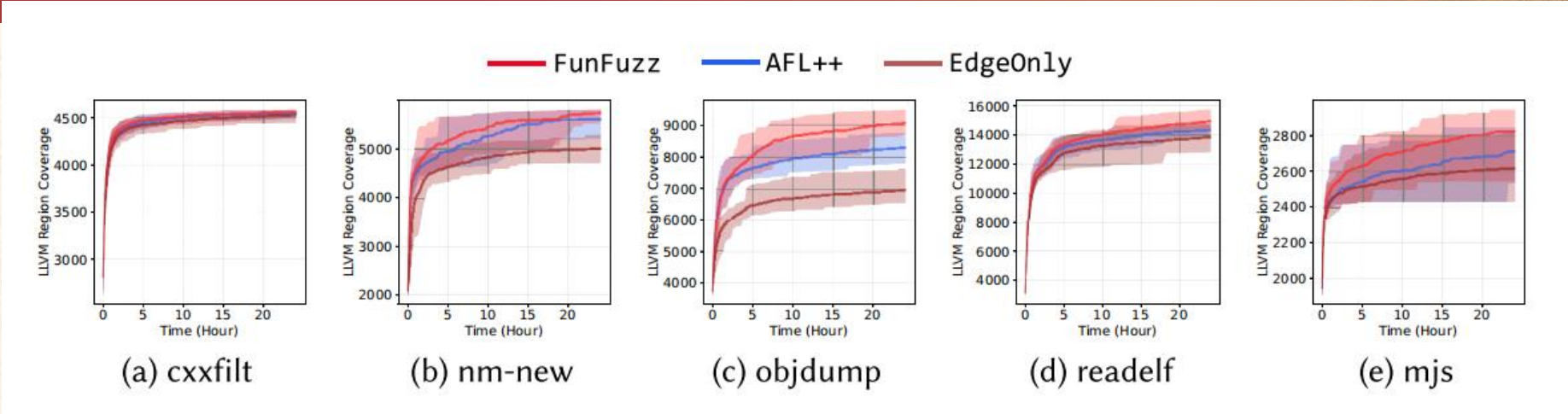
FunFuzz FunFuzz-static AFL++



FunFuzz FunFuzz-static AFL++



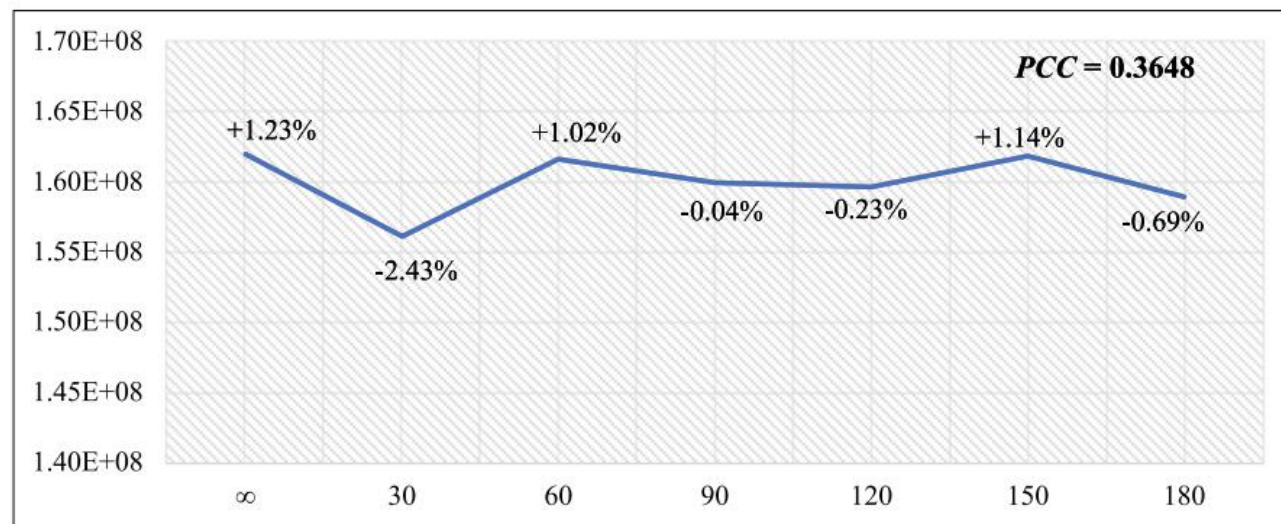
模糊处理过程中两个函数的动态FS值波动



TID	Target	AFL++	FUNFUZZ		EdgeOnly	
		#Reg.	#Reg.	Δ (%)	#Reg.	Δ (%)
T01	cxxfilt	4546.2	4557.2	0.2	4528.2	-0.4
T02	nm-new	5609.1	5731.7	2.2	5006.2	-10.8
T03	objdump	8297.1	9080.2	9.4	6963.7	-16.1
T04	readelf	14333.1	14959.0	4.4	13862.1	-3.3
T07	mjs	2709.6	2824.5	4.2	2617.7	-3.4

FS导向的估计调度策略被证明远比简单的基于边缘覆盖的估计更有效。与一个仅使用边缘覆盖进行能量调整的变体（FUNFuzz-EdgeOnly）相比，FUNFUZZ取得了压倒性优势。EdgeOnly在所有目标上表现甚至不及AFL++。这强有力地说明，忽略函数重要性、仅凭覆盖数量进行调度是无效的，甚至是有害的。FS所提供的“质”的维度信息对于指导测试至关重要。

实验结果分析——RQ5 Fun Period的影响



δ (Second)	∞	30	60	90	120	150	180
∞	—	0.1212	0.4727	0.3447	0.3075	0.5205	0.2730
30	0.1212	—	0.0003	0.0010	0.0036	0.0002	0.0036
60	0.4727	0.0003	—	0.0890	0.1041	0.9097	0.0113
90	0.3447	0.0010	0.0890	—	0.5708	0.0140	0.1620
120	0.3075	0.0036	0.1041	0.5708	—	0.0113	0.6776
150	0.5205	0.0002	0.9097	0.0140	0.0113	—	0.0004
180	0.2730	0.0036	0.0113	0.1620	0.6776	0.0004	—

The cells indicating “no significant difference” (i.e., p-value > 0.05) are colored in gray.

Mann-Whitney U test是一种**非参数统计假设检验方法**，用于评估两个独立数据集之间是否存在显著差异。该检验在模糊测试评估中被广泛采用，主要用于检测两组数据（如覆盖率结果）之间的差异程度。当两个数据集的p值 ≤ 0.05 时，认为它们存在显著差异。若计算出的p值 > 0.05 ，则认为证据不足，无法判定存在显著差异。

如图所示，在**不同的 δ 设置下**（从30秒到180秒），FUNFUzz的吞吐量（以平均总执行次数衡量）保持高度稳定，波动范围极小（-2.43% 到 +1.23%）。

如表所示的统计检验表明，除了极短的30秒周期外，其它设置与不使用FS计算的原始AFL++在**吞吐量上**并无显著差异。

这完美证实了其惰性计算与双共享内存基础设施的高效性，它成功地将动态FS更新的开销降至最低，确保了核心fuzzing循环的流畅运行。



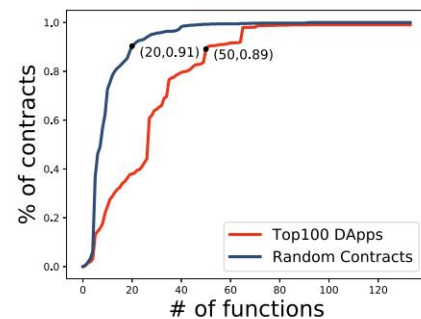
PART 02 ▶

FunFuzz: A Function-Oriented Fuzzer for Smart Contract Vulnerability Detection with High Effectiveness and Efficiency TOSEM 2024

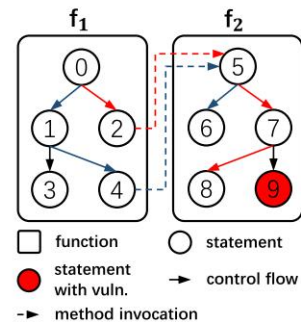
- 智能合约的漏洞分布具有其特殊性——它们往往集中在**少数特定功能函数**中（如涉及跨合约调用、随机数生成或以太币转账的函数），而非均匀散布于全部代码中。这一现象与传统软件漏洞的随机性分布形成鲜明对比。现有的覆盖导向型模糊测试工具（如sFuzz）在**处理函数数量繁多、逻辑交织复杂**的合约时，容易陷入对**非关键函数的过度探索**，难以有效触及深层的脆弱点。

- 将定向模糊测试应用于智能合约仍面临两大挑战
其一，如何自动化地识别出**“风险函数”**
其二，智能合约的测试用例是以交易序列为单位而非单一输入，
因此需要设计专门的种子选择、能量分配与变异策略，
以适应其**多交易、状态依赖的特性**。

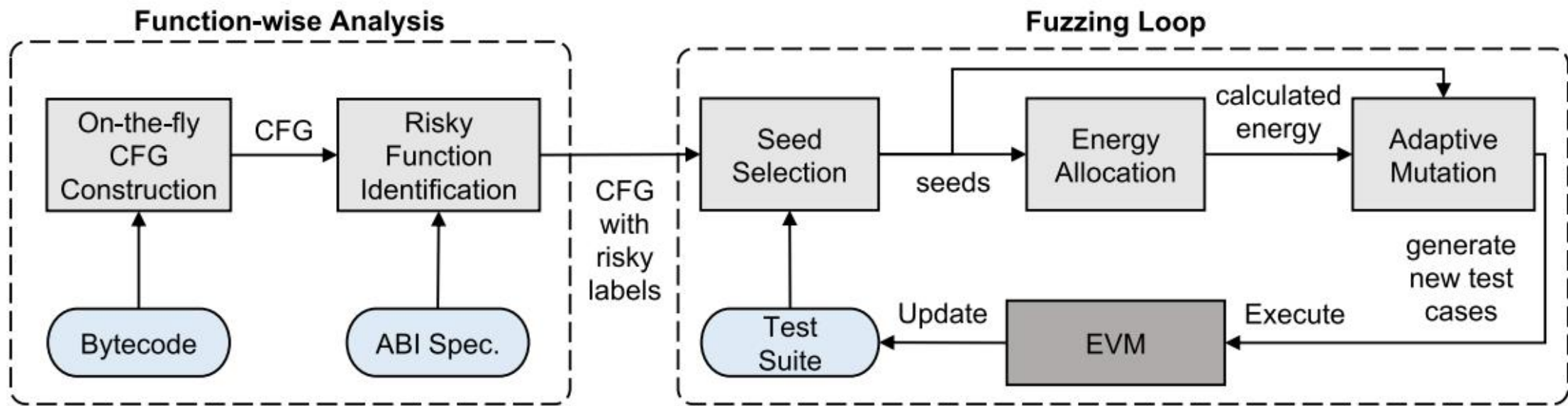
现有方案如生成式fuzzer虽尝试通过预定义交易模式缩小探索范围但仍难以在复杂合约中平衡效率与效果。



(a) The cumulative distribution of function numbers in real-world smart contracts (DApps). As can be seen, smart contracts in top popular DApps are more complicated.



(b) An example of coverage-guided fuzzing. Due to the complexity of the program code, fuzzers may hardly trigger the vulnerable statement.



FunFuzz的核心方法建立在对智能合约漏洞分布特性的深刻洞察之上
其设计理念在于通过定向模糊测试技术，将有限的测试资源精准投放到**高风险函数**中
从而在保证检测效果的同时显著提升效率。
该方法体系主要由两大流程构成：函数导向的**静态分析**与**动态**模糊测试循环，
其中动态循环又整合了种子选择、能量分配和自适应变异三大创新策略。

Vulnerability Type	Description	Critical Opcodes
Reentrancy	Allowing an untrusted contract to repeatedly invoke the calling contracts [19]	CALL
Gasless send	Usage of calling contracts with limitation of the gas [16]	CALL
Timestamp dependency	Using the timestamp of blocks as random numbers [1]	TIMESTAMP
Blocknumber dependency	Using the number of blocks as random numbers [1]	NUMBER
Exception disorder	Exceptions of calling contracts are not checked [16]	INVALID and CALL
Delegatecall	Allowing other contracts to change the runtime bytecode [2]	DELEGATECALL
Freezing Ether	Relying on calling the code of other contracts to send ether[16]	DELEGATECALL
Suicidal contract	A contract can be arbitrarily destroyed [26]	SELFDESTRUCT

漏洞描述及对应的关键操作码

基于智能合约字节码，结合应用二进制接口（ABI）信息，重构控制流图并恢复函数级语义。由于智能合约编译后函数边界信息缺失，FunFuzz利用函数签名哈希动态解析跳转目标从而确定每个公共函数的入口点。

随后，通过静态遍历函数内的操作码，识别与八大类漏洞（如重入、区块依赖、异常无序等）相关的键操作码（例如CALL、TIMESTAMP、DELEGATECALL等）。

只要函数包含这些高风险模式，即被标记为“风险函数”，而非风险函数则被修剪，避免后续测试资源的浪费。这种基于操作码的模式识别，使得FunFuzz能够以较低开销精准定位潜在脆弱点，为定向测试奠定基础。

核心技术——种子选择策略

动态循环的核心是**种子选择**策略，其目标是优先处理更接近未覆盖风险函数的测试用例。

具体而言，距离计算分为两个层次：

首先是分支级距离，它针对单个未覆盖分支；其次是聚合后的多目标距离，它综合考量所有相关分支。

在分支级距离的计算中，FunFuzz动态监控每个测试种子的执行轨迹，重点捕捉风险函数中涉及**条件判断**的操作码（如EQ、GT、LT等）。

每当种子执行到此类分支点，系统会提取栈顶的操作数（例如比较指令中的两个值）

并根据分支条件（如相等性判断）计算一个近似度量。

例如，对于一条基于相等性检查的分支，如果实际值与目标值差异较大，则距离值较高；

反之，若值接近，则距离值较低。

在分支距离的基础上，FunFuzz进一步引入**多目标距离**来应对实际场景中多个未覆盖分支并存的情况。

该距离函数被设计为随未覆盖分支数量的增加而递减，这反映了统计学上的一个基本洞察：目标点越多，随机变异命中某一目标的概率自然提升。

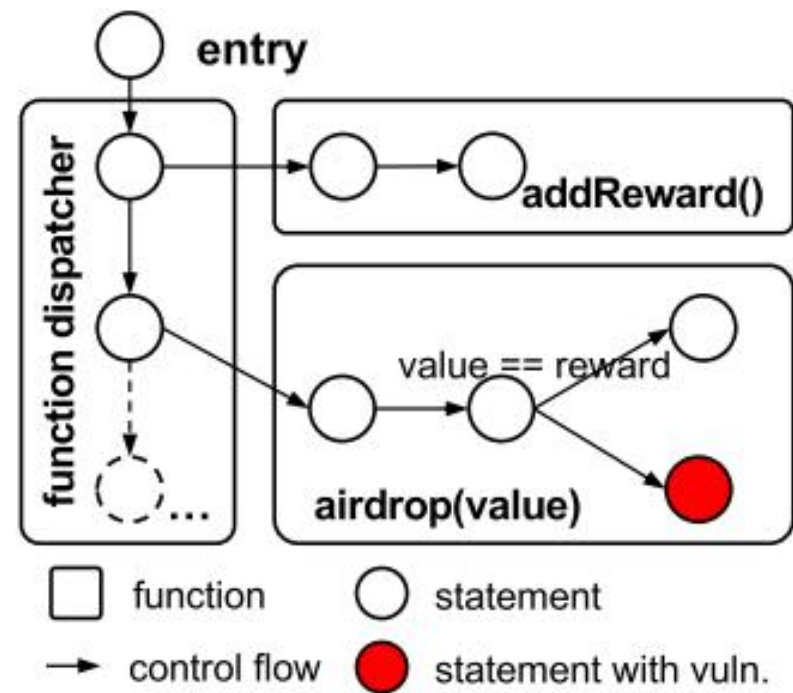
通过这种聚合，FunFuzz无需为每个分支单独维护距离指标，而是用一个统一值表征种子的整体优先度。

这种多目标处理不仅减少了计算开销，还避免了资源分散，确保fuzzer能持续聚焦于最具潜力的种子。

每一轮迭代开始时，系统会计算当前种子池中每个种子的多目标距离，并**优先选择距离最短**的候选者进入变异阶段。这种选择机制类似于一种“进化压力”，驱使测试种群逐步逼近风险函数的核心区域。

核心技术——能量分配策略

```
1 contract Airdrop {  
2   uint256 award = 0;  
3  
4   function addReward() public {  
5     award += 0x1111;  
6   }  
7  
8   function airdrop(uint256 value) public {  
9     if(value == award){  
10      msg.sender.call.value(value)();  
11    }  
12  }  
13 }
```



种子选定后，FunFuzz通过能量分配策略动态调整每个种子的变异预算，以平衡探索与利用。该方法受模拟退火算法启发，将变异操作分为**细粒度**（如单比特翻转）和**粗粒度**（如字典替换）两类。FunFuzz根据种子的归一化距离和测试时间，计算一个接受细粒度变异的概率：距离越短、时间越长，概率越高。能量值随之以指数方式增长，确保接近目标的种子获得更多精细变异机会。例如，在图所示的Airdrop合约案例中，FunFuzz通过**能量分配**快速生成能触发漏洞的参数组合而传统方法可能因过度探索非关键函数而失效。

Data Type	Occupied Space	Actual Space
int<M>/uint<M>	256-bit	M-bit
fixed<M>x/<N>ufixed<M>x<M>	256-bit	M-bit
bytes<M>	256-bit	8M-bit
address	256-bit	160-bit
function	256-bit	192-bit

能量分配策略包含两个模块：变异方法选择和类型感知变异。

在**变异方法选择**中，FunFuzz根据函数标签（风险或非风险）差异化应用变异操作——对于**风险函数**，同时使用细粒度和粗粒度变异；对于**非风险函数**，仅采用粗粒度变异以避免资源浪费。

类型感知模块则利用ABI中的参数类型信息（如uint256、address），避免对填充零等冗余数据进行无效突变。例如，对于uint8类型参数，FunFuzz仅突变其实际占用的8位空间，而非整个256位，大幅减少无意义探索。

FunFuzz以C++构建，集成了定制化的以太坊虚拟机（EVM）用于动态执行与测试预言检测。

整个流程遵循遗传算法框架，通过迭代的选择、分配与变异，使测试种群逐步演化至理想状态。

最终，FunFuzz通过结合轻量静态分析与定向动态测试，实现了在复杂合约中高效挖掘漏洞的目标

Algorithm 1: The Fuzzing Loop of *FunFuzz*.

The initial test suite, S ;

The set of risky functions, F_r ;

for *reach timeout or trigger exception* **do**

$S' = \text{SELECT}(S, F_r)$;

for s in S' **do**

$E = \text{ALLOCATE}(s)$;

for i from 1 to E **do**

$s' = \text{MUTATE}(s)$;

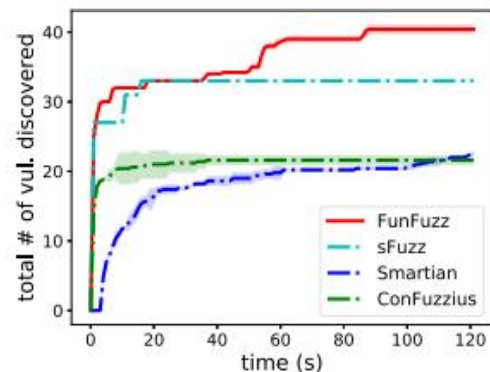
$S \leftarrow s'$;

end

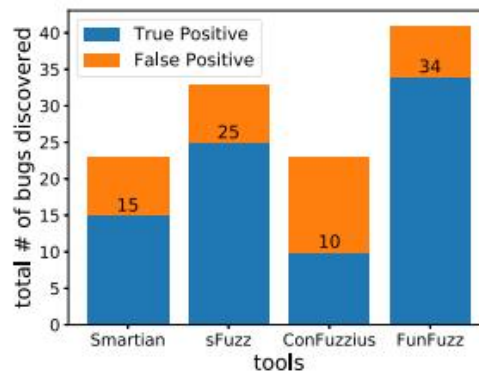
end

end

实验结果——RQ1在复杂合约中发现更多漏洞



(a) Vulnerabilities found by comparison tools over time.



(b) True positives and false positives of comparison tools.

所有工具在165个高复杂度合约上运行2分钟，记录其发现的漏洞数量随时间的变化趋势。如图(a)所示，代表FunFuzz的红色曲线始终位于其他曲线之上，表明其在相同时间内能触发**更多的漏洞**。特别是在测试时间超过60秒后，FunFuzz的优势进一步扩大，这说明其定向策略能够更有效地触及那些深藏在合约代码中的复杂漏洞，而其他工具则可能将时间**浪费在探索非关键函数**上。

对工具报告的结果进行人工核实后，True Positive（真正例）的数量更能说明问题。如图(b)所示，FunFuzz发现了最多经过确认的真实漏洞，其数量分别是sFuzz的1.4倍、Smartian的2.3倍以及ConFuzzius的3.4倍。

实验结果——RQ2以更短的时间达成目标

Contract Name	Vulnerability ID	Smartian [10]	sFuzz [25]	ConFuzzius [34]	FunFuzz
SmartBillions	BD	NA	Within 2 s	Within 2 s	13.2 s
FiftyFlip	BD	NA	NA	7.1 s	33.6 s
LedgerChannel	RE	NA	NA	NA	NA
daoPOLSKAtokens	ED	NA	Within 2 s	Within 2 s	Within 2 s
PandaCore	ED	NA	NA	NA	7 h 3.8 s

效率评估关注的是工具**发现漏洞的速度**。在Deffectiveness数据集上的整体表现已显示FunFuzz更快（图(a)曲线斜率更陡）。定量分析表明，FunFuzz触发相同数量漏洞的速度分别是sFuzz的6.7倍、Smartian的60倍和ConFuzzius的60倍。

为了进一步检验在极端复杂合约上的表现，研究人员在Defficiency数据集的5个合约上进行了长达12小时的测试。

如表所示，FunFuzz在大多数情况下都表现优异。尤其值得注意的是，FunFuzz成功检测出了4个Smartian**未能发现的漏洞**，并且在PandaCore合约中，它是唯一一个在12小时内发现漏洞的工具。虽然在两个合约（SmartBillions和FiftyFlip）上，由于静态分析的开销，FunFuzz的检测速度略慢于ConFuzzius（差距在30秒内）

平均发现漏洞数量上，FunFuzz的效率仍是sFuzz的3.0倍和ConFuzzius的2.0倍。这证明了FunFuzz的策略在面对高复杂度合约时具有显著的时间优势。

实验结果——RQ3各策略的贡献度

分别移除了函数导向分析、种子选择、能量分配和自适应变异策略，然后在相同条件下进行测试。

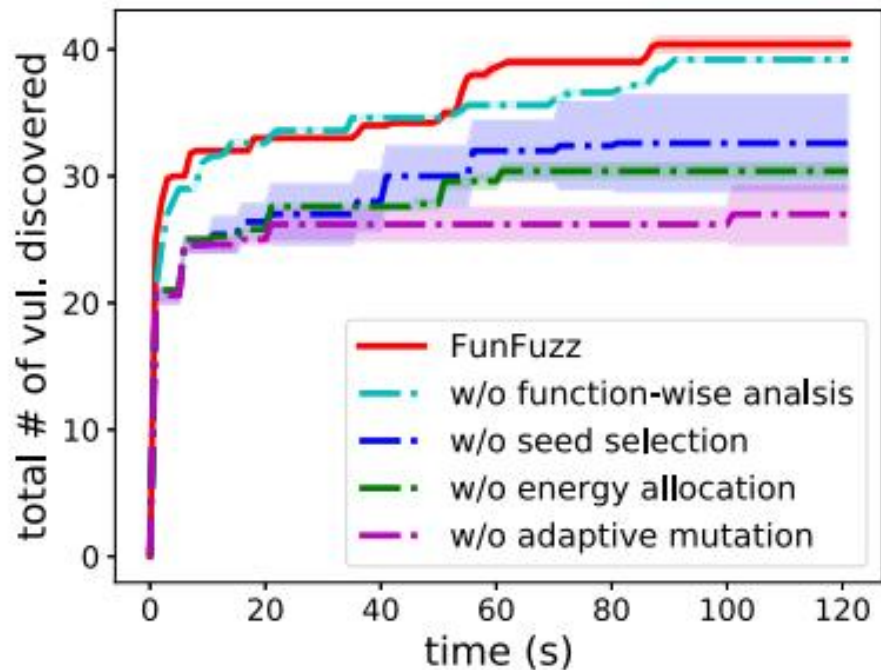
结果如图(c)所示，完整的FunFuzz（红色曲线）性能最优。

移除任何一策略都会导致性能下降，这证实了每个组件都是不可或缺的。

其中，移除自适应变异策略的变体性能最差，凸显了智能变异对效率的关键影响。

定量来看，完整版FunFuzz发现漏洞的速度比这些变体快1.4倍到60倍不等，并且在2分钟内能多发现3.1%到50%的漏洞。

该实验确凿地证明了“通过策略组合缩小探索范围”这一核心思想的有效性。



(c) Vulnerabilities found by *FunFuzz* variants.

文通过一个真实的FoMo3DFast合约案例，生动展示了传统覆盖导向型fuzzer的**局限性**以及FunFuzz解决方案的**优越性**。

该合约存在一个“以区块值作为时间代理”的漏洞，攻击者需按顺序调用 register()和 airdrop()函数才能利用。

对于覆盖导向的fuzzer，如果一个测试用例是 {register(""), airdrop()}，它可能会**优先选择能触发新路径**（如调用 bid()函数）的变异，从而过度探索这个非关键的 bid()函数。

而FunFuzz的解决方案是：首先通过**静态分析**识别出包含TIMESTAMP操作码的airdrop()函数为风险函数；在种子选择时，会优先考虑那些能改变register()参数以使airdrop()分支条件更易满足的种子；再通过能量分配和自适应变异，**高效生成能成功利用漏洞的测试序列** {register("abc"), airdrop()}。

```
1 contract FoMo3DFast {
2     mapping (address => bool) isPlayer;
3     uint256 pool;
4
5     function register(string _nameString) isHuman() public payable {
6         uint256 length = bytes(_nameString).length;
7         require(length <= 32 && length > 3);
8         ...
9         isPlayer[msg.sender] = true;
10    }
11
12    function bid(uint256 value) public payable {
13        require(value != 0);
14        ...
15        pool += value;
16    }
17
18    function airdrop() public {
19        if(isPlayer[msg.sender]){
20            uint256 seed = uint256(keccak256(abi.encodePacked(block.timestamp)));
21            ...
22        }
23    }
24 }
```




PART 03 ▶

TCP-Fuzz: Detecting Memory and Semantic Bugs in TCP Stacks with Fuzzing USENIX 2021

正确实现一个TCP协议栈极具挑战性。这主要源于TCP协议本身功能丰富，涉及可靠传输、拥塞控制等多个复杂机制，并且遵循一个包含多种状态与状态转换的复杂模型。此外，协议还需要处理各种可能出现的异常情况。开发人员在实现过程中难免会引入错误，这些错误通常可分为**内存错误**和**语义错误**两大类。

通过对三个主流TCP协议栈（FreeBSD TCP、mTCP、TLDK）在2017年至2019年间的提交记录进行实证研究发现，高达**87%**的缺陷修复提交是针对**语义错误**，而非**内存错误**。语义错误通常指违反了RFC规范中定义的行为逻辑，或存在其他逻辑不一致性。例如，一个著名的语义错误（CVE-2019-11478）允许攻击者通过特定的SACK序列导致Linux内核TCP栈拒绝服务。这一发现凸显了现有测试方法在检测语义错误方面的不足，因为常用的内存错误检测工具（如ASan、MSan）对这类**逻辑错误**无能为力。

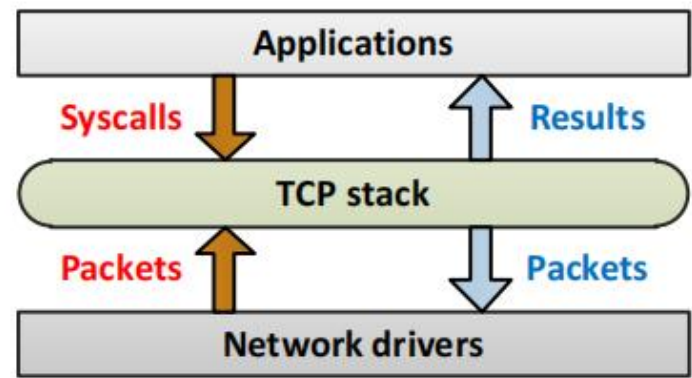
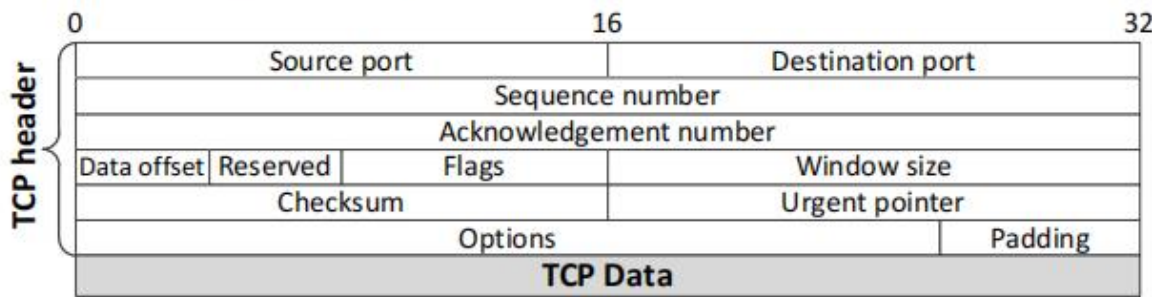


Figure 1: Inputs and outputs of TCP stack.



TCP数据包格式

测试TCP协议栈本身也面临巨大困难。

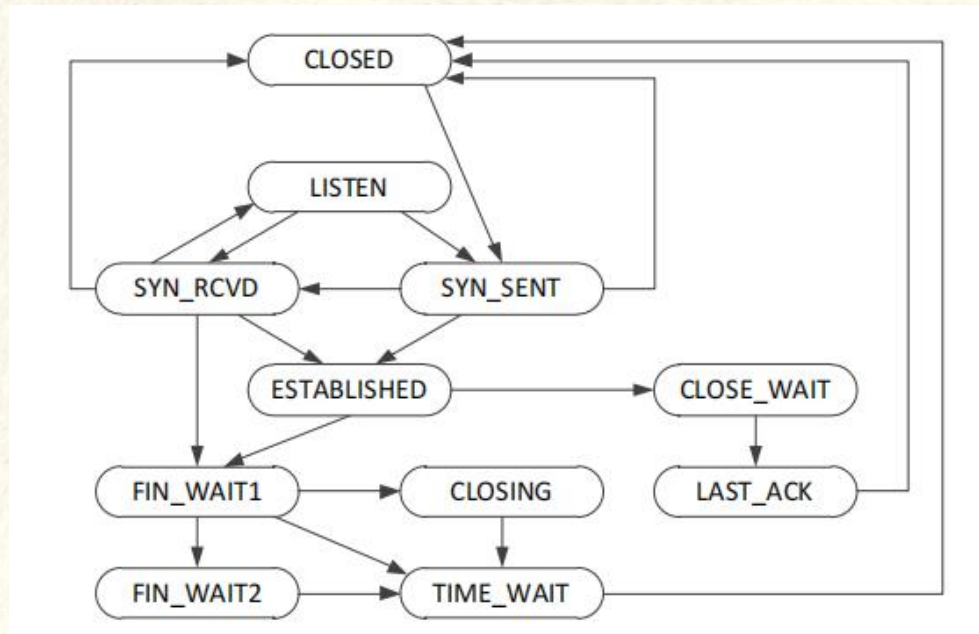
首先，TCP协议栈的输入是二维的，它同时接收来自应用程序的系统调用和来自网络驱动程序的网络数据包，并且这些输入之间存在严格的依赖关系。

例如，**系统调用之间**（如必须先调用socket再调用bind）、**数据包之间**（如序列号必须连续）以及**系统调用与数据包之间**（如accept系统调用需要等待三次握手的完成）都存在**依赖**。

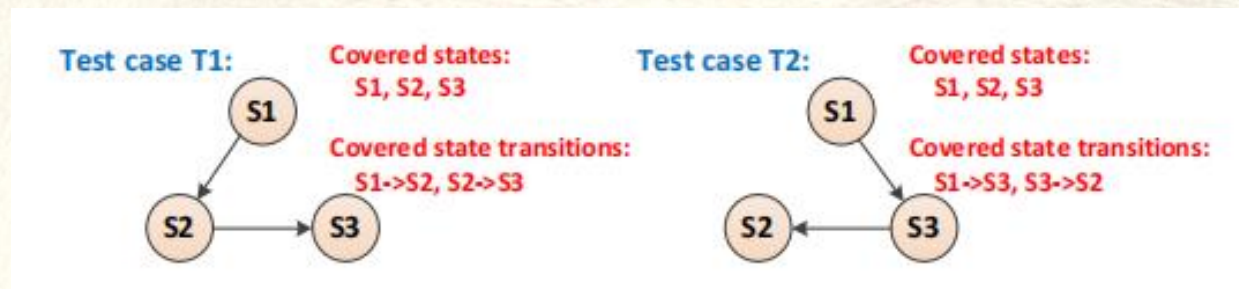
如果测试用例不满足这些依赖，便会被协议栈直接丢弃，无法触发深层的代码逻辑。其次，TCP协议栈的运行基于一个复杂的状态机模型。

仅仅覆盖不同的状态是远远不够的，覆盖状态之间的转换路径对于触发复杂错误往往更为关键。

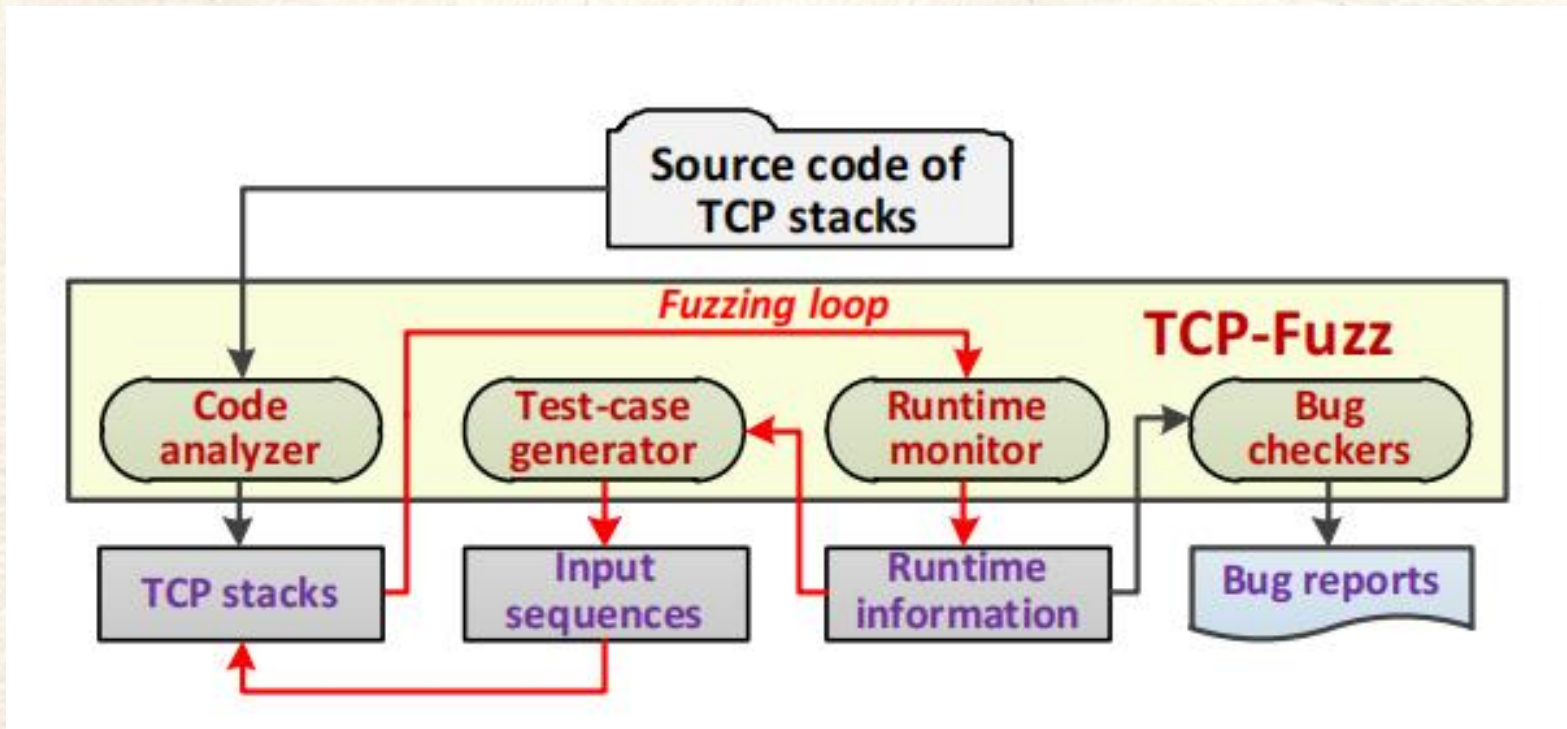
然而，现有的针对应用层协议的模糊测试技术存在明显局限：它们通常只生成**数据包序列**，忽略了**系统调用及其依赖关系**；它们使用**代码覆盖率**作为反馈指标，无法有效引导测试探索不同的**状态转换路径**；它们依赖的内存错误检测器无法捕捉语义错误。



TCP协议栈的基本状态模型



覆盖状态和状态转换的示例。



提出的三个关键技术创新，这些技术共同构成了TCP-Fuzz框架的基础，旨在系统性地解决TCP协议栈测试中的难点。

该方法不仅重新定义了测试用例的生成方式，还引入了新的覆盖度指标和错误检测机制，从而显著提升了对内存错误和语义错误的探测能力。



输入序列突变

CP协议栈的输入包含**系统调用**和**数据包**，二者之间存在复杂的**依赖关系**，例如系统调用之间的**顺序依赖**（如必须先调用socket再调用bind）、数据包之间的**序列号依赖**以及系统调用与数据包之间的**交互依赖**（如accept调用需等待握手包）。

如果测试用例忽视这些依赖，生成的输入会被协议栈直接丢弃，无法有效覆盖代码路径。

为此，TCP-Fuzzi设计了一种基于依赖的测试用例生成策略。

该策略在突变输入序列时，会考虑三种依赖关系（系统调用-系统调用、数据包-数据包、系统调用-数据包），并实现了15条具体的依赖规则（如确保数据包序列号连续）。

突变类型包括删除、添加、替换和修改等，以确保生成的序列既符合协议规范，又能通过小概率违反依赖来测试异常处理逻辑。

这种方法显著提高了测试用例的有效性，使模糊测试能够深入协议栈的核心逻辑。

转换引导的模糊测试方法，解决传统代码覆盖率无法有效捕捉状态转换的问题。

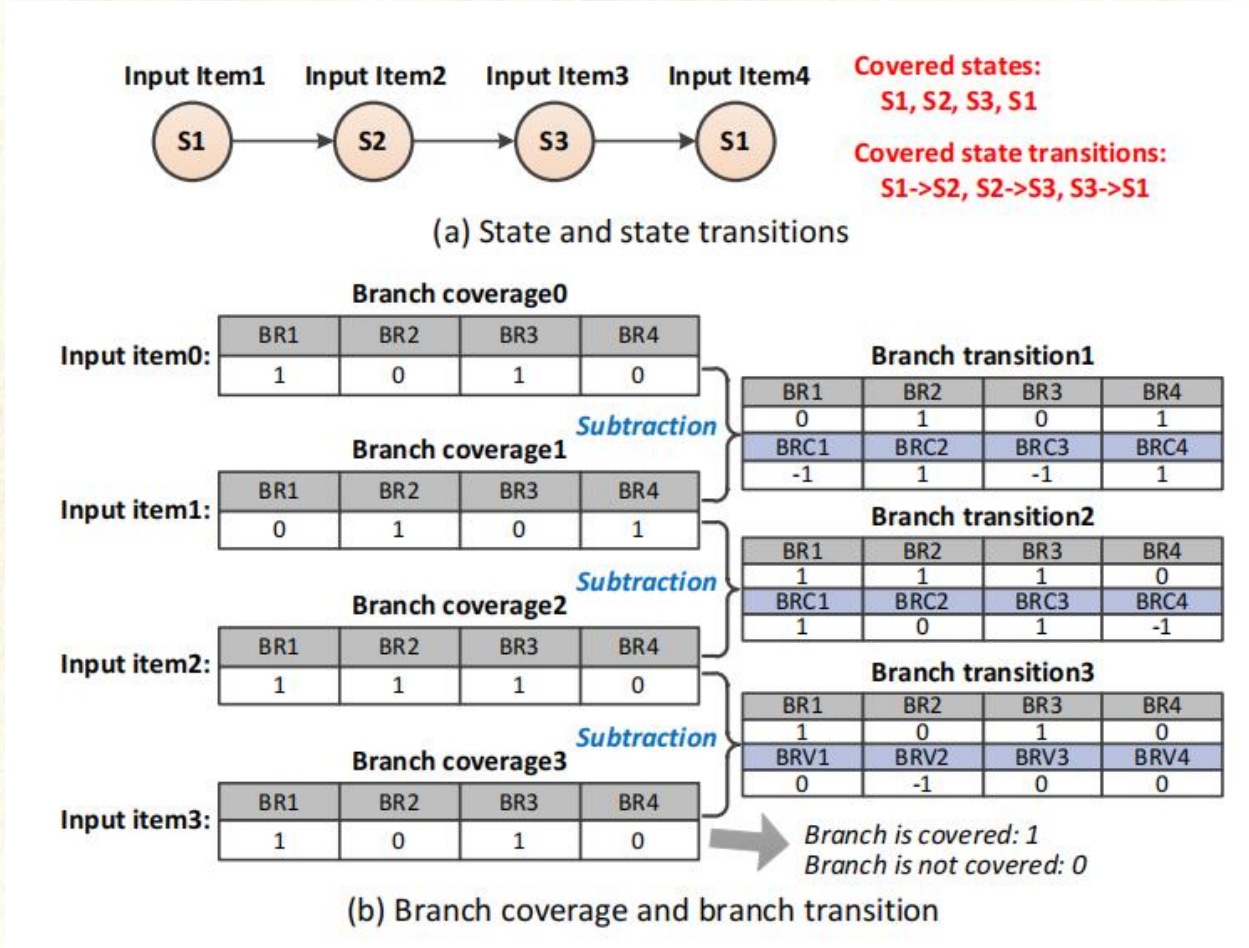
TCP协议栈的行为高度依赖于状态转换（如从建立连接到数据传输的切换），而仅覆盖代码分支（代表状态）不足以触发深层次错误。

TCP-Fuzz引入了一个名为“**分支转换**”的**新覆盖指标**。该指标是一个向量，不仅记录当前输入项（如一个数据包或系统调用）执行后覆盖的代码分支，还计算当前分支覆盖向量与前一个输入项分支覆盖向量的差值，从而刻画状态之间的转换路径。

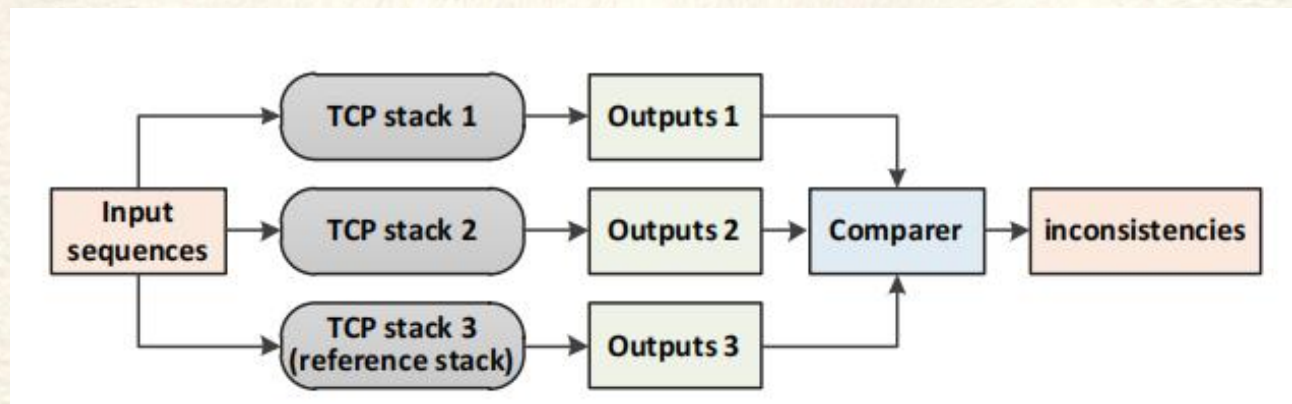
例如，即使两个测试用例覆盖了相同的代码分支，**但若它们以不同顺序触发这些分支**，其分支转换向量会不同，从而被识别为有趣的测试用例。

这种方法使模糊测试能**优先探索新的状态转换路径**，提高了对**复杂语义错误的覆盖概率**。

TCP-Fuzz在运行时收集分支覆盖信息，动态计算分支转换，并以此作为反馈驱动测试用例的变异和选择。



分支转换示例



差分检测器

设计了**差分检查器**，专门用于检测语义错误。

由于语义错误（如违反RFC规则）通常不涉及内存访问异常，传统检测器（如ASan）无法发现。

TCP-Fuzz利用差分测试原理

通过比较多个TCP协议栈（如Linux TCP、FreeBSD TCP等）在相同输入下的输出（包括系统调用返回值和响应数据包）来识别不一致性。

这些不一致性往往暗示某些协议栈存在语义错误，例如对协议选项处理不当或返回错误码。

该检查器可扩展且不引入运行时开销，建议以经过充分测试的内核级协议栈（如Linux TCP）作为参考，来验证较新的用户级协议栈。尽管需要人工分析不一致的根本原因（如对照RFC文档），但这种方法大幅降低了**语义错误检测**的复杂度。

核心技术——部署方案

这些关键技术被集成到一个完整的TCP-Fuzz框架中。

框架包括

代码分析器（使用Clang进行插桩以收集分支覆盖）

测试用例生成器（基于分支转换反馈和依赖策略生成Pack etdrill脚本）

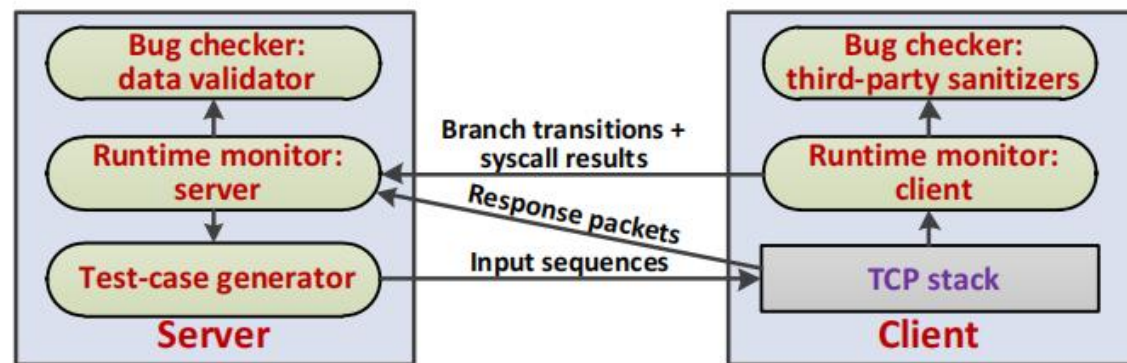
运行时监控器（收集输出和覆盖信息）

错误检查器（结合第三方检测器、数据验证器和差分检查器）

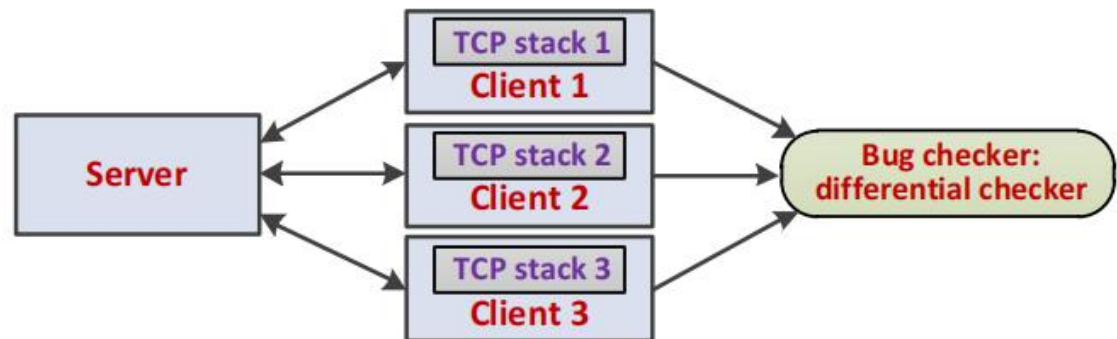
框架采用客户端-服务器部署模式

支持同时对多个协议栈进行测试

实现了对内存错误和语义错误的协同检测。



(a) Communication between a server and a client



(b) Communication between a server and multiple clients

TCP-Fuzz的服务器-客户端部署方案

实验设置基于五个广泛使用的TCP协议栈，包括三个用户级栈（TLDK、F-Stack、mTCP）和两个内核级栈（FreeBSD TCP和Linux TCP）部署在客户端-服务器模式下，每个用户级栈测试48小时，并使用ASan等工具辅助检测。详细阐述实验结果，涵盖测试覆盖度、错误发现、错误影响分析以及与现有工具的对比。

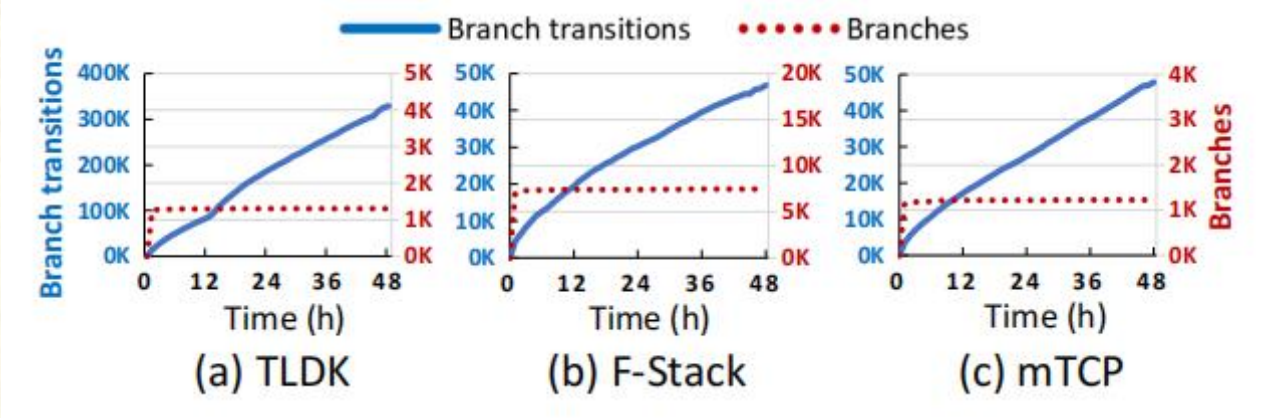
Type	TCP stack	Version	LOC
<i>User-level</i>	TLDK [59]	v2.0	15K
	F-Stack [19]	Commit 8d21adc	25K
	mTCP [26]	Commit 0463aad	18K
<i>Kernel-level</i>	FreeBSD	v12.1	171K
	Linux	v5.6	169K

测试的TCP堆栈的基本信息

实验结果——测试覆盖度与运行时结果

Stack	Testing coverage		Found bugs	
	Branch	Transition	Memory / Semantic	Confirmed / Fixed
TLDK	1.3K	329.4K	2 / 26	28 / 19
F-Stack	7.5K	46.8K	1 / 6	6 / 1
mTCP	1.2K	47.9K	5 / 9	0 / 0
FreeBSD	-	-	0 / 6	5 / 2
Linux	-	-	0 / 1	1 / 1
Total	10.0K	424.1K	8 / 48	40 / 23

TCP堆栈模糊测试的结果



覆盖的分支和分支转换

实验首先关注TCP-Fuzz在测试覆盖度方面的表现。通过代码插桩，TCP-Fuzz收集了分支覆盖和分支转换覆盖数据。结果显示，分支转换的数量远多于分支，这表明TCP协议栈的状态转换路径比静态状态更为复杂，验证了分支转换作为反馈指标的有效性。

覆盖度的增长趋势显示，TCP-Fuzz在测试初期快速覆盖新分支，而在后期虽分支覆盖增长放缓，但分支转换仍持续增加。这体现了转换引导方法的优势：它能探索更多状态转换路径，从而触发深层错误。图展示了三个用户级栈在测试时间内分支和分支转换覆盖的增长情况，可见分支转换的持续上升趋势，而分支覆盖在后期趋于平稳。

实验结果——错误发现与统计分析

TCP-Fuzz共发现了**56个真实错误**，包括8个内存错误和48个语义错误。这些错误分布在所有五个测试栈中：TLDK有28个错误（5个内存错误、23个语义错误），F-Stack有12个错误（2个内存错误、10个语义错误），mTCP有10个错误（1个内存错误、9个语义错误），FreeBSD TCP和Linux TCP作为参考栈，分别发现了4个和2个语义错误。错误报告后，40个错误被开发者确认，其中23个已修复。错误发现过程显示，**内存错误**全部由**ASan**检测到，而语义错误主要通过**差分检查器**（46个）和**数据验证器**（2个）发现。数据验证器检测到的错误尤为危险，因为它们直接导致数据传输不正确，破坏了TCP的基本可靠性。

输出不一致性分析是语义错误检测的关键。TCP-Fuzz报告了**15.1k**个输出不一致案例，通过人工分析RFC文档和栈执行轨迹，识别出48个唯一语义错误。不一致性大多源于相同根本原因，例如多个栈对同一输入的处理差异。仅有一个不一致案例被认定为良性，涉及FIN包后正常包的处理，RFC未明确定义，因此不同栈行为不同（如FreeBSD丢弃包，而Linux重置连接）。

对发现的56个错误进行了**可靠性及安全性影响评估**。结果显示，6个语义错误（主要与RFC 5961相关）易受中间人攻击，可能被利用于连接劫持；9个错误（4个内存错误、5个语义错误）会导致数据损坏，影响传输完整性；8个错误（4个内存错误、4个语义错误）可能引发崩溃或拒绝服务，降低系统可用性；21个语义错误造成功能错误（如异常断开连接）；12个语义错误降低通信效率（如错误丢包）。这些影响突显了TCP栈错误的严重性，尤其是语义错误虽不直接引起崩溃，但可能带来安全风险。

实验结果——错误发现与统计分析



Generated test case in form of a Packetdrill script

```
.....
+0 write(4, ..., 5000) = 5000
+0.1 > . 1:5001(5000) ack 1
+0 < . 1:1(0) ack 1001 win 257
+0.1 close(4) = 0
+0.1 < F. 2001:2001(0) ack 5002 win 257
+0.5 < F. 1:2001(2000) ack 5002 win 257 // Use after free!
```

FILE: TLDK/src/lib/libtle_l4p/tcp_rtx.c

```
int rx_fin(...) {
    .....
    if (s->tcbs.snd.fss >= s->tcbs.snd.nxt &&
        si->ack == (uint32_t)s->tcbs.snd.fss) {
        .....
        + s->tcbs.snd.una_offset = 0;
        + empty_tq(s); // FREE the data of s->tx.q
    }
    .....
}
```

FILE: TLDK/src/lib/libtle_l4p/tcp_txq.h

```
void tcp_txq_rst_next_head(...) {
    struct rte_ring *r = s->tx.q;
    .....
    offset = s->tcbs.snd.una_offset;
    if (offset) {
        m = (struct rte_mbuf*)
            (_rte_ring_get_data(r)[r->cons.tail & r->mask]);
        data_len = m->data_len - PKT_L234_HLEN(m); // USE
    }
    .....
}
```

(a) Use-after-free issue in TLDK

Generated test case in form of a Packetdrill script

```
.....
+0 < S 0:0(0) win 12336
+0 ~ +1 > S. 0:0(0) ack 1 <mss 1460>
+0.1 < . 1:1(0) ack 1 win 25710 <TS val 100 ecr 0,eol,eol>
+0.1 > R 1:1(0) win 0 // RST is sent?
```

FILE: FreeBSD/src/sys/netinet/tcp_synccache.c

```
int synccache_expand(...) {
    .....
    + // If timestamps were not negotiated during SYN/ACK and a
    + // segment with a timestamp is received, ignore the
    + // timestamp and process the packet normally.
    + // See section 3.2 of RFC 7323.
    + if (!(sc->sc_flags & SCF_TIMESTAMP) &&
        (to->to_flags & TOF_TS)) {
        + if ((s = tcp_log_addrs(inc, th, NULL, NULL))) {
        +     log(LOG_DEBUG, ...);
        +     free(s, M_TCPLOG);
        +     s = NULL;
        + }
    }
    .....
    - // If timestamps were not negotiated during SYN/ACK they
    - // must not appear on any segment during this session.
    - if (!(sc->sc_flags & SCF_TIMESTAMP) &&
        (to->to_flags & TOF_TS)) {
        - if ((s = tcp_log_addrs(inc, th, NULL, NULL))) {
        -     log(LOG_DEBUG, ...);
        -     goto failed;
        - }
    }
    .....
}
```

(b) RFC 7323 violation in FreeBSD TCP

Generated test case in form of a Packetdrill script

```
.....
+0 < . 1:1001(1000) ack 1
+0.1 read(4, ..., 1000) = 1000
+0 < . 501:1501(1000) ack 1
+0.1 read(4, ..., 500) = 500 // returns 0?
```

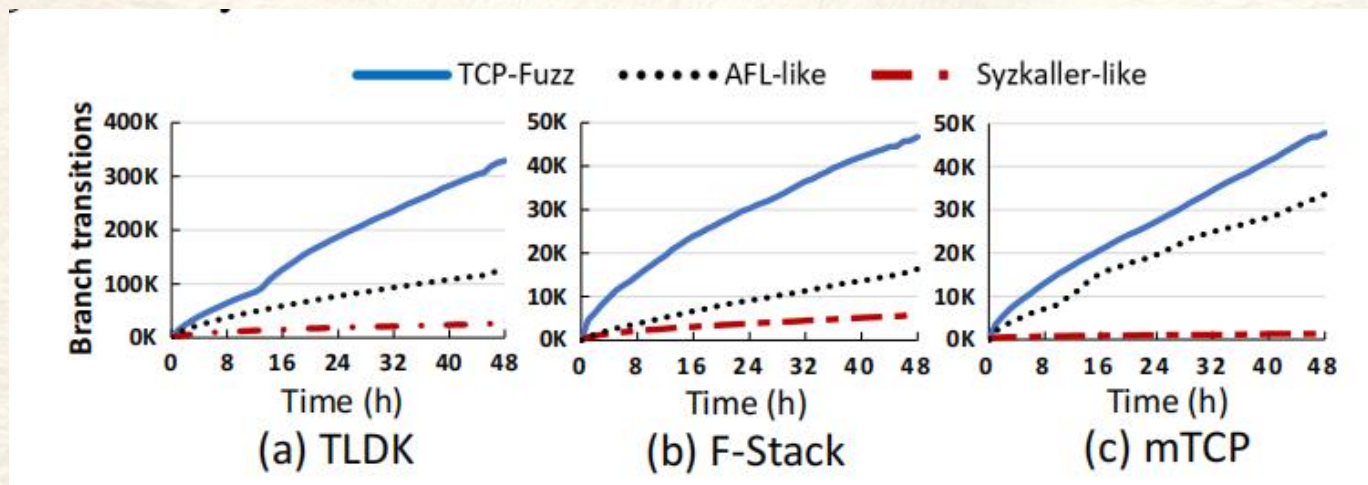
FILE: mtcp/src/tcp_ring_buffer.c

```
int RBPut(...) {
    .....
    + if (cur_seq < buff->head_seq) {
    +     len -= buff->head_seq - cur_seq;
    +     data += buff->head_seq - cur_seq;
    +     cur_seq = buff->head_seq;
    + }
    .....
    + if (GetMinSeq(buff->head_seq, cur_seq) != buff->head_seq)
    +     return 0;
    .....
}
```

(c) RFC 793 violation in mTCP

图展示了TCP-Fuzz发现的三个具体错误示例，包括一个TLDK中的use-after-free错误、一个FreeBSD TCP中的RFC 7323违反错误，以及一个mTCP中的RFC 793违反错误，这些错误均通过Packetdrill脚本生成的测试用例触发。

实验结果——与现有工具的对比



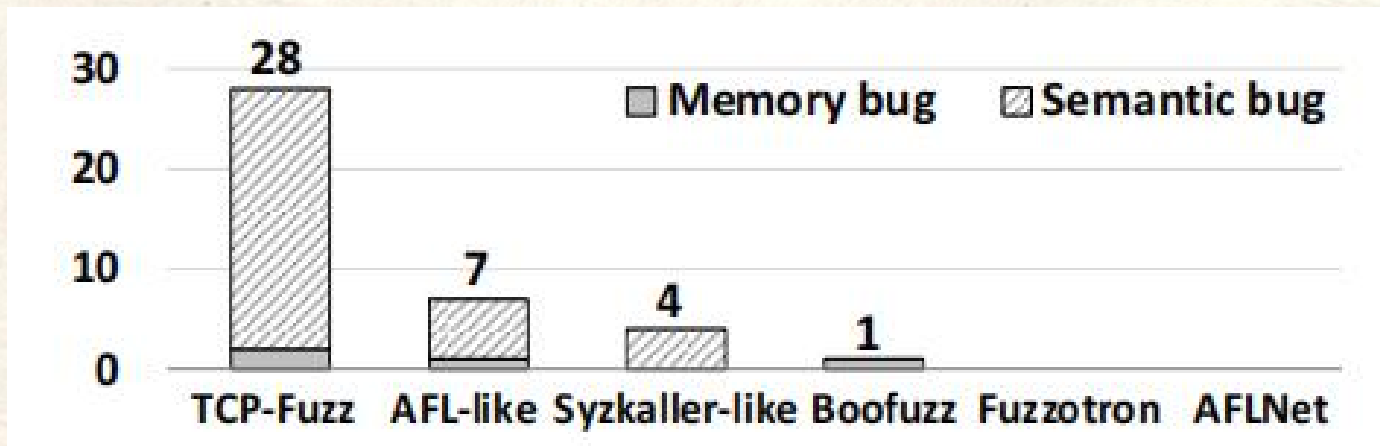
为验证TCP-Fuzz的先进性，作者将其与多种现有模糊测试工具进行了对比。

首先，与AFL-like和Syzkaller-like工具比较：AFL-like工具仅生成数据包序列，Syzkaller-like工具仅生成系统调用序列，两者均基于代码覆盖反馈。

实验显示，TCP-Fuzz在TLDK上覆盖的分支转换远多于这些工具（例如，TCP-Fuzz覆盖约50,000个分支转换，而AFL-like仅覆盖约20,000个，Syzkaller-like覆盖约10,000个），表明二维输入生成和分支转换反馈的有效性。

图对比了测试覆盖度，可见TCP-Fuzz的覆盖增长更快且更持续。

实验结果——与现有工具的对比



TLDK中发现的漏洞比较

在**错误发现方面**，TCP-Fuzz在TLDK中发现了28个错误，而AFL-like工具仅发现7个，Syzkaller-like工具仅发现4个；TCP-Fuzz覆盖了所有这些错误，并额外发现17个遗漏错误。与协议模糊测试工具（如Boofuzz、Fuzzotron和AFLNet）对比：这些工具仅生成数据包，且依赖代码覆盖，结果Boofuzz发现1个空指针解引用，Fuzzotron和AFLNet未发现任何错误。TCP-Fuzz则通过其综合方法发现了27个额外错误。图直观展示了在TLDK中各类工具**发现的错误数量对比**，突显TCP-Fuzz的优越性。



PART 04 ▶

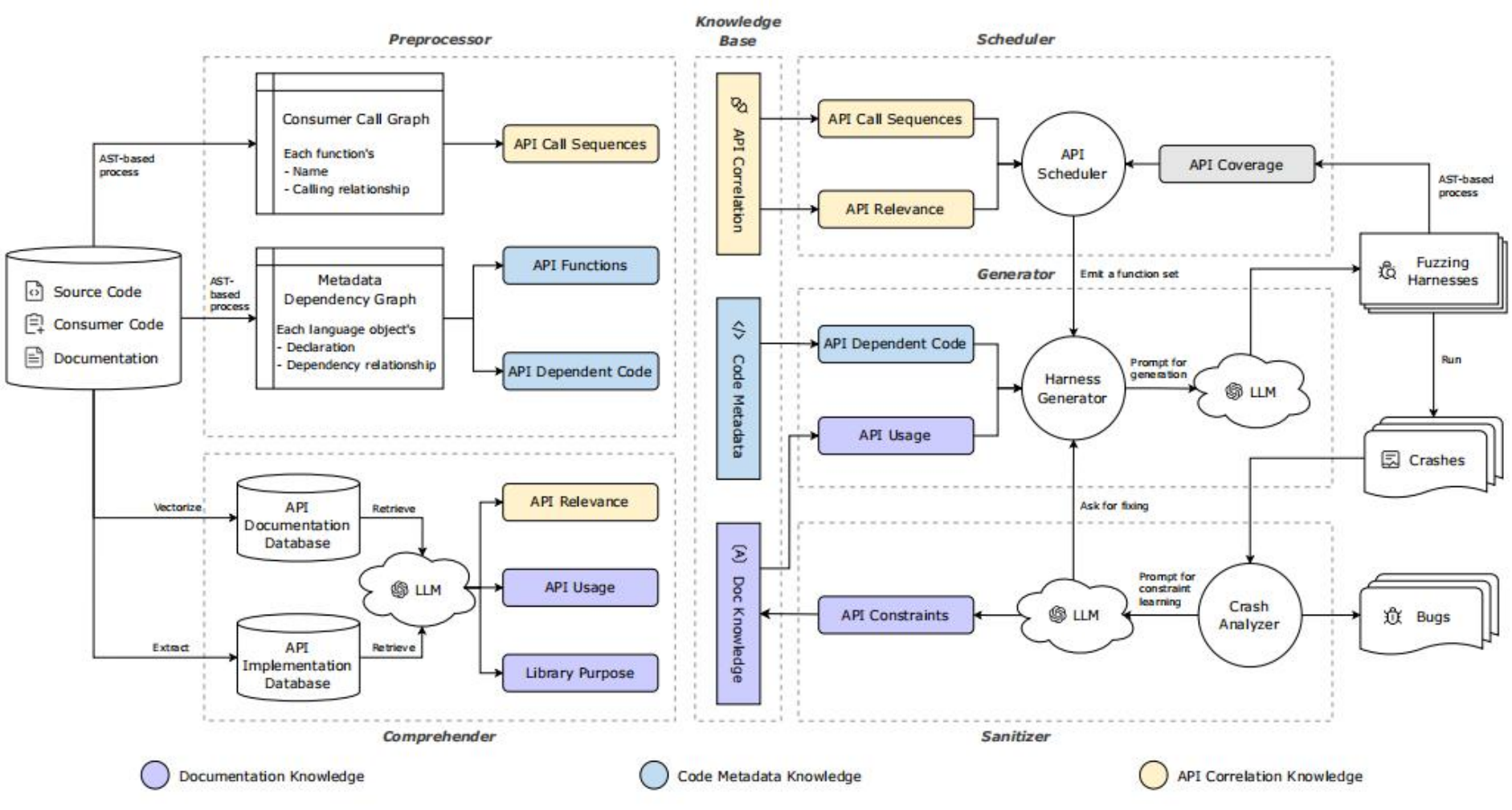
PromeFuzz: A Knowledge-Driven Approach to Fuzzing Harness Generation with Large Language Models CCS 2025

随着软件系统的日益复杂，库（Libraries）的重要性不断提升，对其进行测试的需求也愈发迫切。与测试完整的应用程序不同，测试库需要专门的“**驱动代码**”。因此，驱动代码的质量直接决定了模糊测试的成败——一个设计拙劣的驱动代码可能无法编译，或者即使能运行，也因违反了API的隐式语义规则而迅速崩溃，无法深入测试库的内部逻辑，导致**代码覆盖率低下**和**大量的误报**。

长期以来，生成有效的驱动代码主要依赖开发者的手动编写，这不仅耗时耗力，且难以覆盖库中所有API。现有的自动化方法往往存在明显局限。一类方法**严重依赖于现有的代码**（如示例程序、单元测试），通过程序分析技术从中提取API调用模式来生成驱动代码。然而，这种方法**受限于可用代码的数量和质量**。当目标库缺乏足够或高质量的消费者代码时，这些方法便难以生成有效的测试序列，尤其无法捕捉API之间复杂的、隐式的依赖关系。另一类方法尝试**使用静态分析来探索和组合API调用序列**，但静态分析往往**难以精准理解深层的语义约束**，盲目修改序列容易破坏API间微妙的依赖，从而产生大量无效的驱动代码和误报。

随着大语言模型（Large Language Models, LLMs）在代码理解和生成方面展现出强大能力，研究人员开始探索利用LLMs来自动生成驱动代码。这类方法（如提到的PromptFuzz、CKGFuzzer等）将**API签名、类型定义甚至调用图等信息**作为**提示**输入给LLM，依靠模型内嵌的编程知识来生成语法和语义上合理的代码。这确实减少了人工干预，并相比传统方法有所改进。但这些LLM驱动的方案仍存在根本性缺陷：它们对**目标库的“知识”获取是不完整的**。LLMs很容易产生“**幻觉**”，生成看似合理但实际上违反了API关键约束的代码。例如，在论现有工具生成的驱动代码因**忽略**了文档中明确指出的缓冲区大小约束而直接崩溃。这种因驱动代码本身错误而非库真实漏洞导致的崩溃，就是典型的误报。

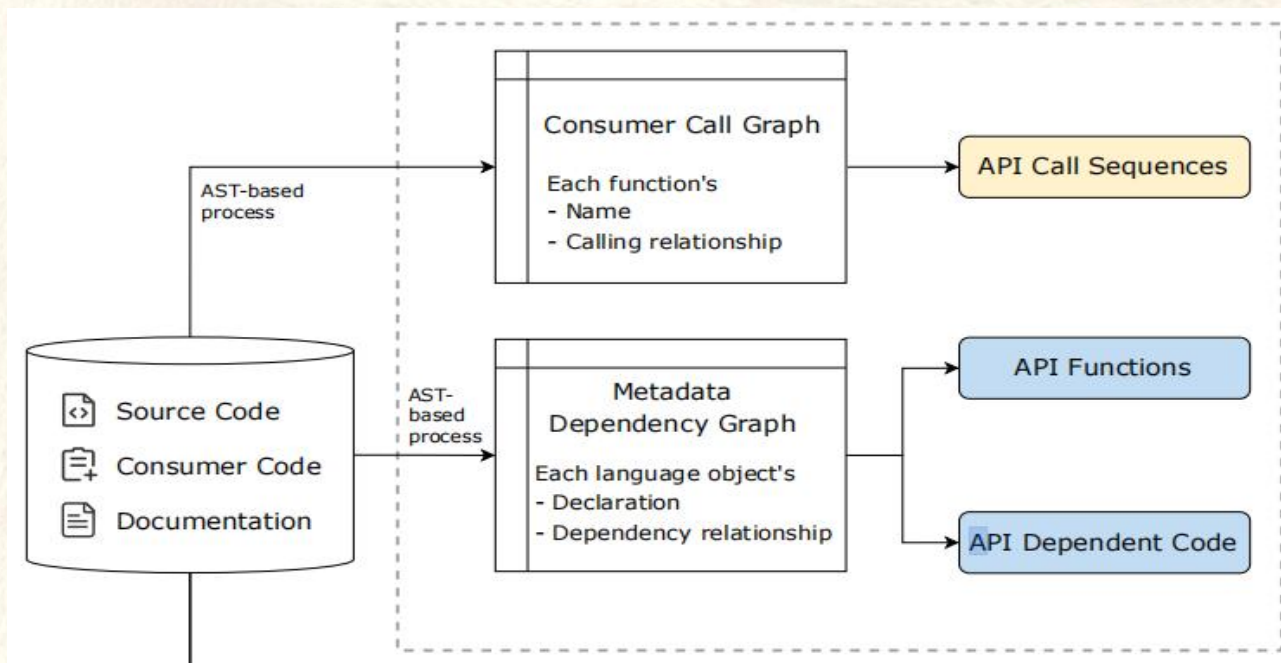
1. 知识库构建：从原始信息到结构化知识
- 预处理模块：提取代码层面的结构化知识
 - 理解模块：提炼语义层面的深度知识
2. 知识驱动的驱动代码生成与优化
- 调度器模块：智能选择API组合策略
 - 生成器模块：在精准知识指导下代码生成
 - 净化器模块：闭环反馈与自我修正



论文框架

预处理模块：提取代码层面的结构化知识

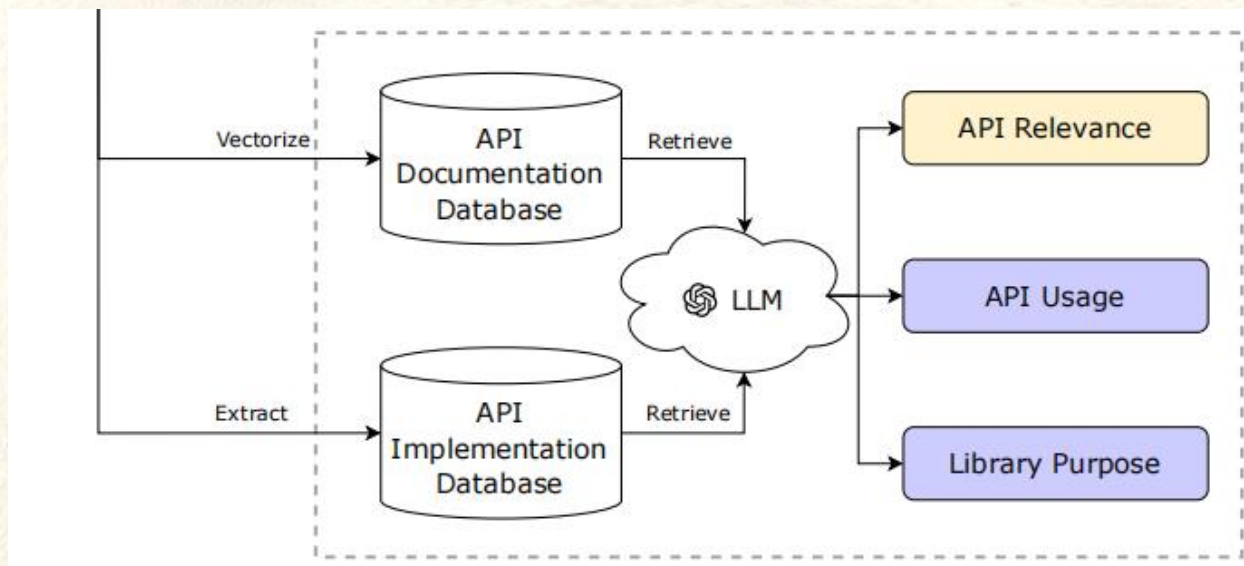
- 功能：该模块使用程序分析技术（基于Clang AST）对库的源代码进行深度解析。
- 元数据依赖图**。它不仅仅提取**API函数签名**，还会识别所有**相关的语言对象**（如类、结构体、枚举、类型定义），并构建它们之间的依赖关系网络。例如，一个API函数依赖于某个类，该类又依赖于其构造函数和某些成员变量。这个图确保了在生成驱动代码时，所有必要的类型定义和初始化代码都能被正确包含，从根本上保证语法正确性。
- 消费者调用图**。从现有的示例代码、单元测试或已有的驱动代码中，提取**真实的API调用序列**。这些序列代表了经过验证的、常见的**API使用模式**，为后续的API调度提供了宝贵的参考。



核心技术——知识库构建

理解模块：提炼语义层面的深度知识

- 功能：该模块旨在理解API“如何正确使用”的语义信息。
- API用法理解：采用**检索增强生成（RAG）**技术。它首先为API的源代码实现和文档内容创建向量数据库。当需要理解某个特定API时，通过检索相关代码片段和文档块，并交由LLM进行总结，从而得到该API的设计目的、参数约束、前置后置条件等关键语义信息。这直接解决了动机案例中因忽略文档约束（如errbuf缓冲区大小）而导致误报的问题。
- API相关性度量：为了生成能协同工作、深度探索库状态的API序列，PROMEFUZZ从三个维度量化API之间的相关性：
 - 类型相关性**：通过参数和返回类型的兼容性判断API间是否存在数据流传递的可能。
 - 范围相关性**：包括“类范围”（同一类的成员方法可操作相同实例状态）和“调用范围”（在消费者调用图中，被同一函数调用的API可能相关）。
 - 语义相关性**：利用LLM根据API的功能描述，判断它们在逻辑上是否相关（例如，“添加属性”和“删除属性”是逻辑上配对的操作）。



核心技术——知识驱动的驱动代码生成与优化

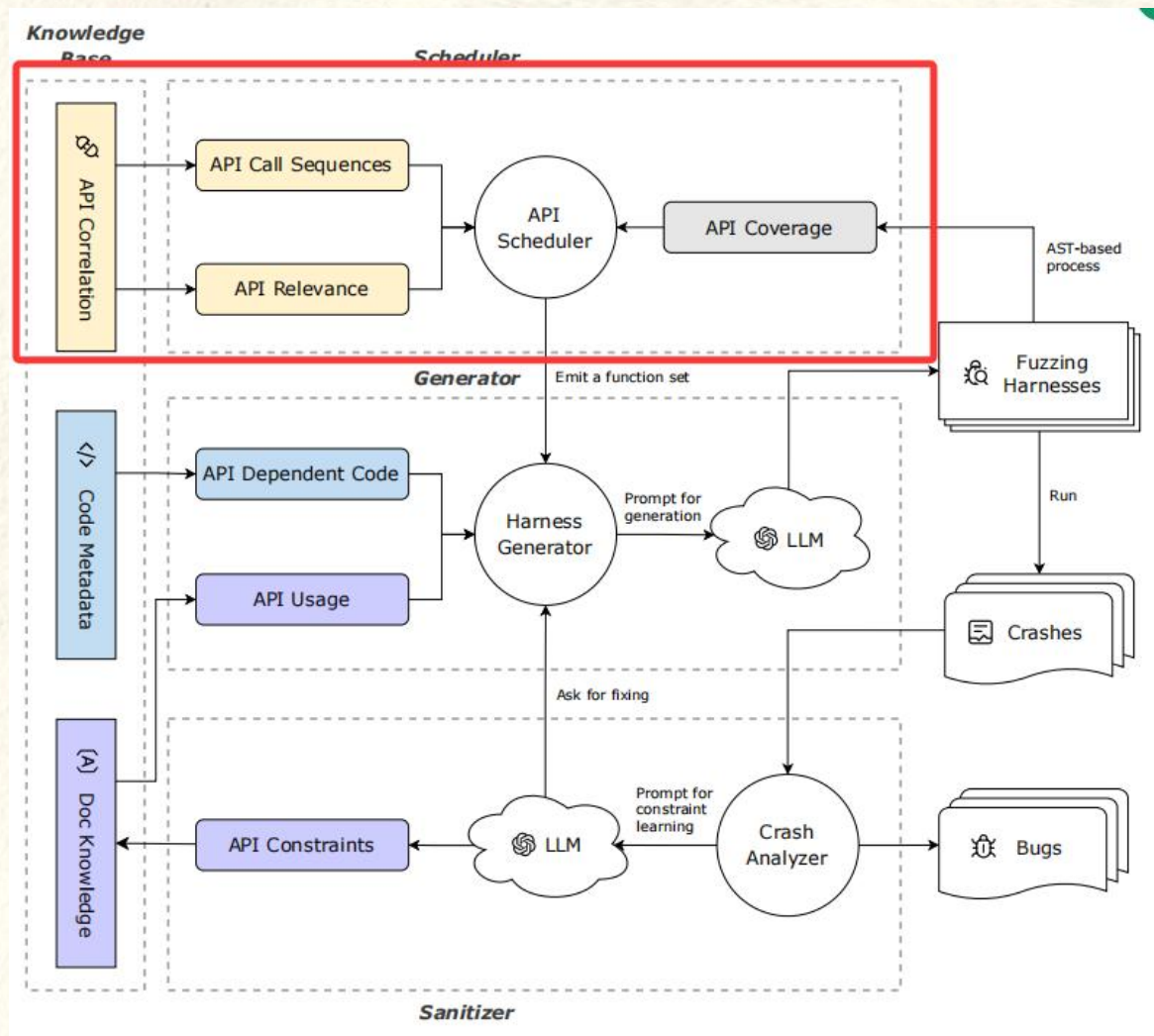
调度器模块：智能选择API组合策略

第一阶段：优先利用专家知识（**现有调用序列**）

调度器并非从头开始构建所有测试组合。它的首要策略是优先利用从高质量源代码（如单元测试、示例程序）中提取出的、由开发者编写的**现成API调用序列**。这些序列被认为是经过验证的、语义正确的API使用模式。只要这样的序列存在，调度器就会直接将其作为一个完整的API集合（记为集合 T ）发送给生成器模块。这确保了在初始阶段就能**快速生成高质量、低误报的驱动代码**。

第二阶段：智能构建新组合（**基于调度分数**）

当所有现成的调用序列都被使用完毕后，调度器便启动其核心的、基于**量化评分**的动态组合构建算法。此过程是一个迭代的贪心算法，旨在逐步构建一个最具潜力的API测试集合。



$$rel_{pq} = \frac{w_{typ} typ_rel_{pq} + w_{scp} scp_rel_{pq} + w_{sem} sem_rel_{pq}}{w_{typ} + w_{scp} + w_{sem}}$$

$$score_p = \frac{w_{cov}(1 - cov_p) + w_{rel} \cdot MinMax(\sum_{f_i \in T} rel_{pi})}{w_{cov} + w_{rel}}$$

当所有现成的调用序列都被使用完毕后，调度器便启动**基于量化评分的动态组合构建**算法。此过程是一个迭代的贪心算法，旨在逐步构建一个最具潜力的API测试集合。

1. 初始化与迭代构建

算法开始时，集合 T 仅包含一个虚拟的起始点 f_0 （其与所有API的相关性为0）。随后，在每一次迭代中，它都会从所有未被选入 T 的API函数库 F 中，选择一个“最佳”的API加入 T ，直到 T 的大小达到预设的上限。

2. 量化相关性与覆盖率

选择“最佳”API的关键在于为一个候选API f_i 计算其调度分数（）。这个分数是以下两个核心指标的加权综合：

覆盖率激励（Exploration Incentive）：鼓励测试覆盖不足的API。其计算为 $1 - cov_i$ ，其中 cov_i 是API f_i 的历史调用频率归一化值。调用越少的API，此项值越高，优先级越高。

关联强度激励（Exploitation Incentive）：鼓励选择与当前集合 T 内API高度相关的候选者。它本身是一个综合指标。

关联强度 rel_{pq} 是三种相关性的加权平均值

rel_{pq} 是整体相关性得分， typ_rel_{pq} 是类型相关性， scp_rel_{pq} 是范围相关性， sem_rel_{pq} 是语义相关性， w 是相关权重

$score_p$ 是函数 f_p 最终调度分数， cov_p 是函数的历史覆盖率（归一化后）， T 是在当前轮次已经被选中的目标API函数集合

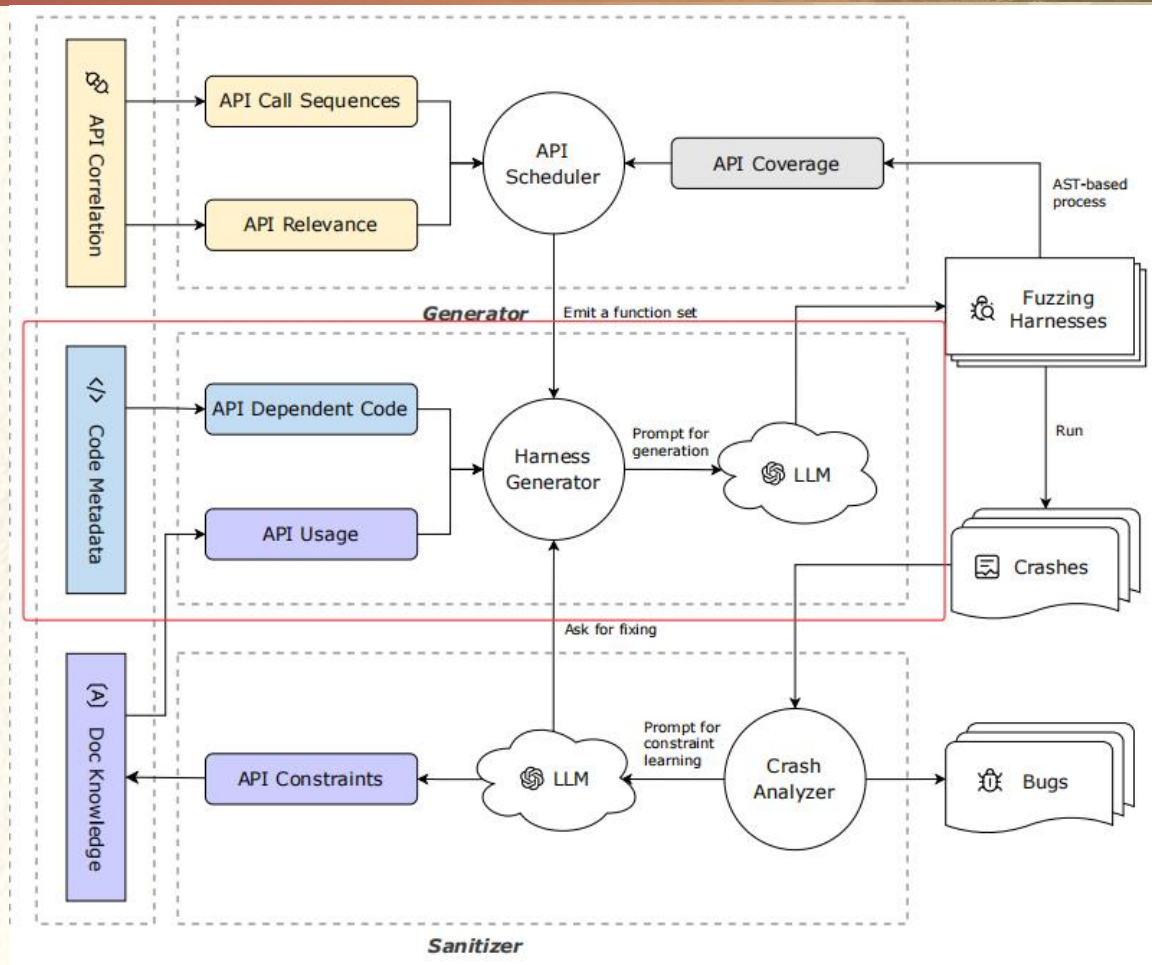
MinMax是归一化函数， w 是相关权重，求和式计算的是**候选函数**与当前**目标集合 T** 中**每一个已有函数的整体相关性的和**

核心技术——知识驱动的驱动代码生成与优化

生成器模块：在精准知识指导下代码生成

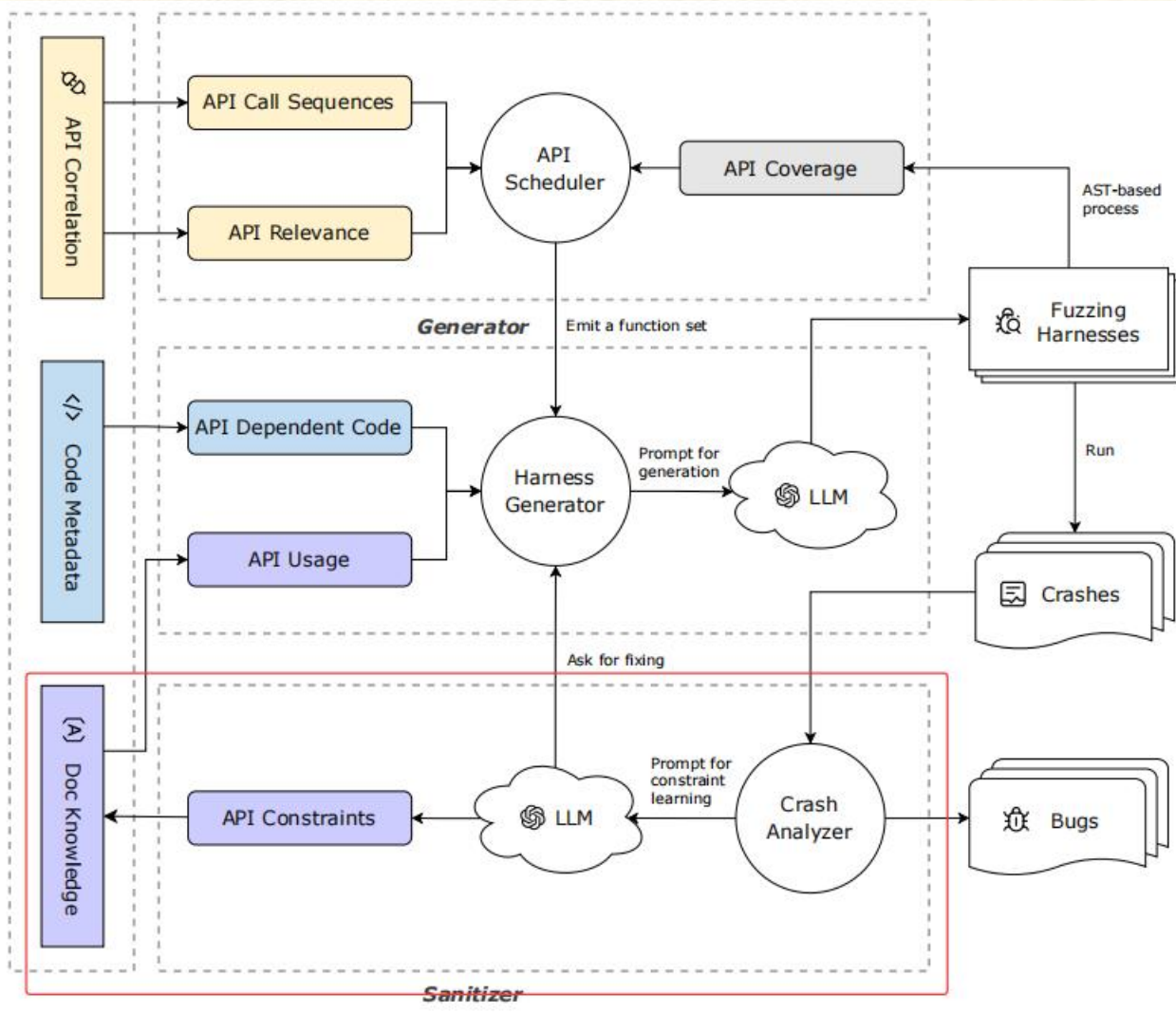
功能：根据调度器选定的API集合，生成具体的驱动代码。

过程：生成器会从构建的知识库中，精准检索出与目标API相关的所有必要信息：包括通过**遍历元数据依赖图得到的依赖代码声明**，以及从理解模块得到的**API用法约束**。将这些结构化的知识作为提示（Prompt）输入给LLM，指令其**生成符合规范的驱动代码**。这种方法确保了生成的代码建立在全面、准确的信息基础上。



净化器模块：闭环反馈与自我修正

- 功能：这是PROMEFUZZ区别于其他方法的关键，它负责处理LLM生成代码时不可避免的错误，形成一个学习闭环。
- 编译错误：直接利用编译器的错误信息反馈给LLM进行修复，相对简单。
- 运行时崩溃分析：当驱动代码在模糊测试过程中发生崩溃时，净化器会启动一个**基于推理模型的崩溃分析器**（例如DeepSeek-R1 or OpenAI o1）。它会综合崩溃报告、驱动代码、相关源码和API约束，深度分析根本原因。
- 快速路径：对于首次运行即发生的崩溃，大概率是API误用，直接尝试修复。
- 全尺度路径：对于运行一段时间后发生的重复崩溃，进行深入分析以判断是库的**真实漏洞**（True Positive）还是驱动代码的误用（False Positive）。如果是误用，分析器会进一步总结出**正确的“API约束”**（例如，“set_value的参数必须以空字符结尾”），并将这条新学到的知识反馈回知识库，用于提升后续生成的质量。



PromeFuzz生成的模糊测试框架总体测试结果。

Tested Library					PROMEFuzz's Cost			Coverage (#Branch / #API)				
Name	Version	LoC	#API	#Branch	Time	Expense	Token	PROMEFuzz	PromptFuzz	CKGFuzzer	OSS-Fuzz-Gen	OSS-Fuzz
c-ares	7978cf7	44K	136	8,237	1h14m	\$0.78	3.8M	6,106 / 113	5,711 / 59	5,677 / 91	3,612 / 28	3,733 / 19
curl	4c50998	187K	96	29,942	31m	\$0.28	1.4M	7,347 / 95	5,006 / 87	6,143 / 91	759 / 27	950 / 7
lcms	04ace9c	46K	358	9,040	52m	\$1.59	11.3M	4,560 / 358	3,672 / 231	3,767 / 227	374 / 32	716 / 31
libjpeg-turbo	36ac5b8	64K	72	10,636	33m	\$0.18	1.0M	4,274 / 71	3,520 / 78	N/A	957 / 14	4,480 / 26
sqlite3	7903db4	180K	270	50,903	1h13m	\$2.44	17.1M	28,225 / 246	26,263 / 160	N/A	17,987 / 17	25,568 / 15
libaom	99fcd81	530K	47	61,360	34m	\$0.19	0.9M	13,697 / 47	13,596 / 45	N/A	11,365 / 20	11,290 / 11
libpcap	2559282	42K	99	5,944	47m	\$0.20	1.2M	3,825 / 89	2,752 / 73	3,358 / 71	502 / 21	3,051 / 10
libvpx	9514ab6	370K	37	32,357	21m	\$0.12	0.8M	5,898 / 37	3,594 / 30	3,578 / 32	3,206 / 14	2,755 / 7
zlib	5a82f71	30K	89	2,720	2h06m	\$0.17	0.9M	2,430 / 89	2,099 / 85	2,343 / 66	1,865 / 34	1,789 / 29
re2	c84a140	28K	79	8,520	31m	\$0.13	0.8M	6,236 / 78	5,222 / 30	N/A	5,408 / 17	6,134 / 36
libmagic	dadc01f	18K	18	6,124	3m	\$0.04	0.2M	3,868 / 18	120 / 9	N/A	2,689 / 8	3,132 / 8
libpng	738f5e7	58K	256	6,999	51m	\$1.10	7.6M	3,538 / 251	3,296 / 228	N/A	881 / 16	2,081 / 27
ngiflib	db19270	1K	7	357	5m	\$0.06	0.2M	351 / 7	283 / 7	323 / 6	40 / 5	N/A
ffjpeg	caade60	2K	40	585	7m	\$0.12	0.4M	571 / 40	375 / 22	N/A	305 / 33	N/A
liblouis	3d95765	38K	30	6,247	10m	\$0.07	0.3M	4,704 / 30	2,220 / 17	N/A	1,723 / 22	3,797 / 5
libtiff	fcd4c86	92K	198	13,528	28m	\$0.49	3.5M	8,096 / 198	7,428 / 136	7,109 / 148	4,983 / 68	5,666 / 13
cjson	12c4bfl	17K	78	924	7m	\$0.11	0.6M	899 / 78	863 / 75	706 / 43	779 / 57	502 / 6
pugixml	06318b0	24K	221	5,957	1h24m	\$0.32	2.0M	3,957 / 221	N/A	N/A	369 / 2	3,385 / 7
tinygtf	a5e653e	193K	65	11,594	8m	\$0.12	0.5M	5,739 / 65	N/A	N/A	1,541 / 2	2,885 / 1
exiv2	04e1ea3	79K	880	44,320	5h46m	\$7.17	43.4M	13,580 / 816	N/A	N/A	8,144 / 7	9,857 / 17
loguru	4adaa18	3K	84	883	10m	\$0.15	0.5M	621 / 66	N/A	190 / 33	233 / 10	N/A
rapidcsv	083851d	6K	49	326	8m	\$0.07	0.3M	205 / 49	N/A	N/A	10 / 8	N/A
SUM			3,209	317,503	18h5m	\$15.89	98.4M	128,727 / 3,062	86,020 / 1,372	33,194 / 808	67,489 / 444	91,771 / 275

实验在22个真实世界的开源项目上进行，涵盖了多种库和规模，与当前最先进的工具（如PromptFuzz、CKGFuzzer、OSS-Fuzz-Gen）以及人工编写的驱动代码（如OSS-Fuzz中的条目）进行了深入比较。实验结果充分展示了PROMEFUZZ的优越性。

PROMEFUZZ展现出了惊人的效率和覆盖度。它成功为95.4%的API函数（总计3062个）生成了测试驱动，产生了2577个驱动代码，其中超过78%是可直接用于模糊测试的有效代码。整个知识构建和生成过程在约18小时内完成，成本可控，平均每个API仅需21秒和0.005美元，体现了其实际可行性。

最关键的性能指标——分支覆盖率上，PromeFuzz取得了压倒性优势。其生成的驱动代码达到的分支覆盖数量是PromptFuzz的1.50倍，是CKGFuzzer的3.88倍，是OSS-Fuzz-Gen的1.91倍，甚至比经验丰富的开发者手动编写的OSS-Fuzz驱动代码还高出1.40倍。

统计检验（Mann-Whitney U test）表明，在绝大多数项目上，这种优势是显著的（p值通常小于0.05）。这种高覆盖率的根源在于其知识驱动的方法能够引导模型生成语义正确的调用，从而直接调用了远多于其他工具的API函数（是PromptFuzz的1.34倍，是OSS-Fuzz的10.5倍），使得测试能够深入库的内部逻辑。

相比之下，其他LLM工具由于缺乏对库约束的深入理解，生成的驱动代码往往因API误用而提前崩溃，无法有效执行。

Table 7: Mann-Whitney U test p-values for branch coverage across 10 runs.

Library	PromptFuzz	CKGFuzzer	OSS-Fuzz-Gen	OSS-Fuzz	w/o Comp	w/o Sched	w/o San
c-ares	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04
curl	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04
lcms	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	1.37E-02
libjpeg-turbo	9.77E-04	N/A	9.77E-04	1.00E+00	9.77E-04	9.77E-04	9.77E-04
sqlite3	9.77E-04	N/A	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04
libaom	6.52E-01	N/A	1.95E-03	2.93E-03	2.16E-01	2.16E-01	1.95E-03
libpcap	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-03	9.77E-04	9.77E-04
libvpx	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	1.95E-03
zlib	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	6.77E-01
re2	9.77E-04	N/A	9.77E-04	9.77E-04	2.73E-02	9.77E-04	9.77E-04
libmagic	9.77E-04	N/A	9.77E-04	9.77E-04	1.16E-01	9.77E-03	1.16E-01
libpng	9.77E-04	N/A	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04
ngiflib	9.77E-04	9.77E-04	9.77E-04	N/A	2.54E-02	9.77E-04	9.77E-04
ffjpeg	9.77E-04	N/A	9.77E-04	N/A	6.25E-02	9.77E-04	9.77E-04
liblouis	9.77E-04	N/A	9.77E-04	9.77E-04	2.16E-01	6.84E-03	1.37E-02
libtiff	1.86E-02	1.95E-03	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04
cjson	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04
pugixml	N/A	N/A	9.77E-04	9.77E-04	4.88E-03	9.77E-04	9.77E-04
tinygltf	N/A	N/A	9.77E-04	9.77E-04	9.77E-04	9.77E-04	9.77E-04
exiv2	N/A	N/A	9.77E-04	9.77E-04	9.99E-01	1.95E-03	9.77E-04
loguru	N/A	9.77E-04	6.25E-02	N/A	9.77E-04	9.77E-04	9.77E-04
rapidcsv	N/A	N/A	9.77E-04	N/A	9.77E-04	9.77E-04	9.77E-04
P <0.05	16	10	21	17	17	21	20

RQ2漏洞发现能力

在测试中，共触发了29次独特的崩溃，经过人工验证，其中26个是真实存在的库漏洞（真阳性），精度高达89.7%。更有22个漏洞已得到库开发者的确认。

尤为重要的是，PromeFuzz发现了25个此前未知的安全漏洞，其中21个已被确认，17个已被修复，更有3个因严重性被分配了CVE标识（如Exiv2库中的CVE-2025-26623，一个use-after-free漏洞）。

这一成果在长期被OSS-Fuzz覆盖的成熟库中取得，更显其价值。

与此形成鲜明对比的是，基线工具虽然触发了更多崩溃（如PromptFuzz触发56个，CKGFuzzer触发64个），但绝大多数都是因API误用导致的假阳性，精度极低（分别为1.8%和6.3%），这会给安全研究人员带来巨大的分析负担。

PromeFuzz的高精度得益于其净化器模块中的崩溃分析器，能够有效区分真实漏洞和API误用。

PromeFuzz、PromptFuzz、CKGFuzzer、OSS-Fuzz-Gen和OSS-Fuzz发现的崩溃

Library	PROME		Prompt		CKG		OSS-Gen		OSS-Fuzz	
	#TP	#FP	#TP	#FP	#TP	#FP	#TP	#FP	#TP	#FP
c-ares	0	0	0	1	0	9	0	2	0	0
curl	0	0	0	1	0	8	0	0	0	0
lcms	0	1	0	2	0	13	0	1	0	0
libjpeg-turbo	0	1	0	4	-	-	0	2	0	0
sqlite3	0	0	0	10	-	-	0	1	0	0
libaom	0	0	0	6	-	-	0	2	0	0
libpcap	0	0	0	1	0	9	0	5	0	0
libvpx	0	0	0	0	0	3	0	1	0	0
zlib	0	0	0	3	0	7	0	1	0	0
re2	0	0	0	1	-	-	0	2	0	0
libmagic	2	0	0	0	-	-	0	0	0	0
libpng	0	0	0	6	-	-	0	0	0	0
ngiflib	2	0	0	6	2	0	0	1	-	-
ffjpeg	4	1	0	0	-	-	0	5	-	-
liblouis	12	0	1	5	-	-	1	1	5	0
libtiff	2	0	0	7	0	6	0	6	0	0
cjson	0	0	0	2	2	1	0	2	0	0
pugixml	0	0	-	-	-	-	0	0	0	0
tinygtf	0	0	-	-	-	-	0	0	0	0
exiv2	1	0	-	-	-	-	0	0	0	0
loguru	0	0	-	-	0	4	0	10	-	-
rapidcsv	3	0	-	-	-	-	0	0	-	-
SUM	26/22	3	1/1	55	4/3	60	1/1	42	5/5	0
Precision	89.7%		1.8%		6.3%		2.3%		100%	

The number of crashes confirmed by the developer is listed in the right of /.

PromeFuzz发现的新现实世界漏洞

Library	ID	Vulnerability Type	Status	P	C	OG	OF
ngiflib	Issue 34	NULL Pointer Dereference	Fixed	×	✓	×	-
	Issue 36	NULL Pointer Dereference	Fixed	×	✓	×	-
ffjpeg	Issue 55	Use Uninitialized Variable	Reported	×	-	×	-
	Issue 57	Buffer Underflow	Reported	×	-	×	-
	Issue 58	Stack Buffer Overflow	Reported	×	-	×	-
	Issue 60	Stack Buffer Overflow	Reported	×	-	×	-
liblouis	Issue 1708	Invalid Free	Fixed	×	-	✓	×
	Issue 1709	Invalid Free	Fixed	×	-	×	×
	Issue 1711	Memory Leak	Confirmed	×	-	×	×
	Issue 1718_1	Memory Leak	Fixed	×	-	×	×
	Issue 1718_2	Memory Leak	Fixed	×	-	×	×
	Issue 1719	Heap Buffer Overflow	Fixed	×	-	×	✓
	Issue 1720	Heap Buffer Overflow	Fixed	×	-	×	✓
	Issue 1721	Heap Buffer Overflow	Fixed	×	-	×	✓
	Security j9qg	Infinite Loop	Confirmed	×	-	×	×
	Security 3wwm	Heap Buffer Overflow	Confirmed	×	-	×	✓
libtiff	Security w63h	Heap Buffer Overflow	Confirmed	×	-	×	×
	Issue 669	Reachable Assertion	Fixed	×	×	×	×
exiv2	CVE-2025-45701	Memory Leak	Fixed	×	×	×	×
	CVE-2025-26623	Use After Free	Fixed	-	-	×	×
libmagic	CVE-2025-55450	Integer Overflow	Fixed	×	×	×	×
	Bug tracker 640	Integer Overflow	Fixed	×	×	×	×
rapidcsv	Issue 186	Out-of-bounds Write	Fixed	-	-	×	-
	Issue 187	Out-of-bounds Write	Fixed	-	-	×	-
	Issue 188	Out-of-bounds Read	Fixed	-	-	×	-

P: PromptFuzz C: CKGFuzzer OG: OSS-Fuzz-Gen OF: OSS-Fuzz

RQ3每个组件的贡献

通过逐一关闭核心模块（如知识理解模块Comprehender、调度器Scheduler、净化器Sanitizer）并重新实验，结果清晰地表明每个组件都不可或缺。

当缺少知识理解模块时，分支覆盖率下降至完整系统的86%，漏洞检测精度暴跌至29.4%，因为模型失去了关键的语义约束指导。

当关闭调度器，转而随机选择API组合时，覆盖率降至完整系统的79%，精度降至20.0%，这证明智能调度对于生成有逻辑的API序列至关重要。

影响最大的是关闭净化器模块，覆盖率骤降至74%，精度更是低至2.7%，这与基线工具的高假阳性率水平相当，凸显了闭环反馈和错误修正机制对于生产可用驱动代码的关键作用。

此外，研究还探讨了调度算法中权重参数的影响，发现过度偏向覆盖率或关联性都会损害整体性能，而论文采用的平衡设置（如覆盖率和关联性权重比为3:1）能取得稳定优良的效果。

Tested Library			PROMeFuzz			w/o Comp			w/o Sched			w/o San		
Name	API	Branch	API	Branch	Query	API	Branch	Query	API	Branch	Query	API	Branch	Query
c-ares	136	8,237	113	6,106	532	94	5,669	631	80	4,793	215	40	5,107	136
curl	96	29,942	95	7,347	226	95	2,738	273	90	4,452	58	63	3,158	78
lcms	358	9,040	358	4,560	480	341	4,174	409	269	3,344	214	313	4,481	273
libjpeg-turbo	72	10,636	71	4,274	159	72	4,005	68	72	3,774	33	62	3,369	64
sqlite3	270	50,903	246	28,225	736	250	22,570	669	191	18,880	296	170	16,191	311
libaom	47	61,360	47	13,697	99	45	13,452	162	45	13,641	19	38	11,799	46
libpcap	99	5,944	89	3,825	115	91	3,758	112	92	3,655	50	83	3,684	100
libvpx	37	32,357	37	5,898	55	37	4,383	52	34	3,108	19	35	3,713	55
zlib	89	2,720	89	2,430	106	89	2,307	82	87	2,019	25	89	2,436	68
re2	79	8,520	78	6,236	86	77	6,205	174	67	6,036	38	69	5,559	59
libmagic	18	6,124	18	3,868	26	18	3,679	38	18	3,551	5	18	3,685	30
libpng	256	6,999	251	3,538	335	251	3,220	423	249	3,038	116	219	2,930	217
ngiflib	7	357	7	351	75	7	347	124	6	302	12	7	287	32
ffjpeg	40	585	40	571	139	39	562	181	35	479	40	29	504	53
liblouis	30	6,247	30	4,704	107	30	4,642	497	20	4,358	42	17	4,457	54
libtiff	198	13,528	198	8,096	486	194	5,714	361	173	6,168	148	185	5,206	221
cjson	78	924	78	899	85	78	890	78	78	810	31	77	882	94
pugixml	221	5,957	221	3,957	197	215	3,918	329	187	3,449	70	203	3,524	252
tinygtf	65	11,594	65	5,739	88	65	3,045	70	65	2,695	18	54	2,794	31
exiv2	880	44,320	816	13,580	1,792	812	14,655	1,458	736	12,691	721	495	11,335	457
loguru	84	883	66	621	185	69	526	311	42	497	102	15	68	64
rapidcsv	49	326	49	205	63	50	193	102	40	117	14	43	169	52
SUM	3,209	317,503	3,062	128,727	6,172	3,019	110,652	6,604	2,676	101,857	2,286	2,324	95,338	2,747

Library	default	w _{cov} ↑	w _{rel} ↑	w _{typ} ↑	w _{scp} ↑	eq1	eq2
curl	7,347	8,032	5,376	8,325	8,548	8,195	5,916
lcms	4,560	4,492	4,570	4,902	4,443	4,418	4,800
sqlite3	28,225	17,702	18,739	21,237	20,667	23,706	21,846
libaom	13,697	21,060	12,690	14,335	15,966	19,724	14,406
libvpx	5,898	7,576	4,224	4,436	3,648	3,857	4,766
libpng	3,538	3,540	3,598	3,464	3,612	3,777	3,567
ffjpeg	571	559	521	555	504	567	575
liblouis	4,704	3,343	4,080	4,796	4,623	4,691	4,990
cjson	899	891	857	884	904	900	894
tinygtf	5,739	2,917	2,901	2,981	2,716	3,113	3,890
SUM	75,178	70,109	57,555	65,913	65,630	72,947	65,650

default denotes $w_{cov} : w_{rel} = 3 : 1$ and $w_{typ} : w_{scp} : w_{sem} = 2 : 3 : 5$
 $w_{cov} \uparrow$ denotes $w_{cov} : w_{rel} = 5 : 1$ and $w_{typ} : w_{scp} : w_{sem} = 2 : 3 : 5$
 $w_{rel} \uparrow$ denotes $w_{cov} : w_{rel} = 1 : 3$ and $w_{typ} : w_{scp} : w_{sem} = 2 : 3 : 5$
 $w_{typ} \uparrow$ denotes $w_{cov} : w_{rel} = 3 : 1$ and $w_{typ} : w_{scp} : w_{sem} = 5 : 3 : 2$
 $w_{scp} \uparrow$ denotes $w_{cov} : w_{rel} = 3 : 1$ and $w_{typ} : w_{scp} : w_{sem} = 2 : 5 : 3$
eq1 denotes $w_{cov} : w_{rel} = 1 : 1$ and $w_{typ} : w_{scp} : w_{sem} = 2 : 3 : 5$
eq2 denotes $w_{cov} : w_{rel} = 1 : 1$ and $w_{typ} : w_{scp} : w_{sem} = 1 : 1 : 1$



恳请各位老师同学批评与指导



汇报人：许嘉星



日期：2025/11/04